

Signer Process Isolation — Design Document

Version: v19.3-design-draft-6

Status: § 7.3.1, § 7.3.4, § 9.4, § 13 APPROVED for implementation. § 7.3.2/ § 7.3.3/ § 7.3.5/ § 7.3.6 unchanged from draft-3 (already approved). **§ 7.3.7 (rate limiter hotfix v19.3.2) NOT YET APPROVED** — operator review of draft-5 caught three remaining issues that are addressed in this draft-6 revision: (1) the `http.Server.prototype.emit` monkey-patch lacked a try/catch with safe-degradation fallback (a single exception in the capture path would crash the server on EVERY request, a strictly worse failure mode than the bug being fixed); (2) the regression tests for v19.3.2 (scenarios 18/19 in `test-vault.ts`) called the pure decision function `resolveTrusted-ClientIp` with synthetic `peerAddress` values — they did NOT exercise the actual monkey-patch nor the AsyncLocalStorage propagation, leaving the riskiest component untested; (3) the coupling of the capture mechanism to `http.Server.prototype.emit('request', ...)` was not documented — a future migration to HTTP/2 or a custom server would silently degrade the rate limiter to the shared `"unidentifiable-socket"` bucket. Draft-6 addresses all three: try/catch added with `peerAddress=null` fallback (degrades to named bucket, never propagates exception); new `scripts/test-request-peer-integration.ts` exercises the REAL monkey-patch + ALS with a live `http.Server` and two distinct loopback source IPs (`127.0.0.1` + `127.0.0.2`, both in 127.0.0.0/8 per RFC 5735) under concurrent requests-in-flight; explicit COUPLING-TO-http.Server comment added to `request-peer-capture.ts` with detection query in § 13.7. **Phase 1 implementation remains BLOCKED on operator re-review of § 7.3.7 — the rate limiter hotfix is NOT yet "shipped" in the sense of being closed.**

Author: engineering

Date: 2026-07-14

Revision history:

- **v19.3-design-draft** (original): process isolation + JSON-RPC + method allowlist. Operator review identified critical gap — no signer-side authorization policy.
- **v19.3-design-draft-2**: adds § 7.3 (Signer-Side Authorization Policy) as Phase 1 BLOCKING prerequisite; rewrites § 9 to operator's stated defaults; adds § 12.2 with clean 150s stress test result + port-collision hypothesis findings; fixes empty-vault unlock bug in `wallet-crypto.ts`.
- **v19.3-design-draft-3**: operator's conditional approval round. Five sign-off prerequisites incorporated: (1) § 9.4 writer lock → writer **lease with TTL** (auto-expires on holder death; fail-closed restart consequence made explicit); (2) § 7.3.4 hash-chained audit log moved from Phase 2 to **Phase 1** (local tamper-evidence is cheap; only remote shipping stays Phase 2); (3) § 7.3.2 + new § 7.3.7 — explicit answer on rate-limit / window-cap scoping: failed-unlock rate limiter is **per source IP** (was global — confirmed DoS vector; the 150s stress test data shows the `operator/lifecycle/interleaver` profiles WERE blocked by the `attacker` profile's cooldown — the bug was manifesting in our own evidence); USD rolling cap remains global (signer = single writer identity, no per-IP rotation possible); (4) § 7.3.1 — explicit confirmation that the price feed URL/source is **fixed at boot env var**, NOT a parameter in the `sign` RPC (same tamper-resistance pattern as the allowlist); (5) new § 13 **Operator Runbook** with step-by-step procedures for allowlist edits, cap changes, and chain expansion.

- **v19.3-design-draft** (original): process isolation + JSON-RPC + method allowlist. Operator re-view identified critical gap — no signer-side authorization policy.
- **v19.3-design-draft-2**: adds § 7.3 (Signer-Side Authorization Policy) as Phase 1 BLOCKING prerequisite; rewrites § 9 to operator's stated defaults; adds § 12.2 with clean 150s stress test result + port-collision hypothesis findings; fixes empty-vault unlock bug in `wallet-crypto.ts`.
- **v19.3-design-draft-3**: operator's conditional approval round. Five sign-off prerequisites incorporated: (1) § 9.4 writer lock → writer **lease with TTL** (auto-expires on holder death; fail-closed restart consequence made explicit); (2) § 7.3.4 hash-chained audit log moved from Phase 2 to **Phase 1** (local tamper-evidence is cheap; only remote shipping stays Phase 2); (3) § 7.3.2 + new § 7.3.7 — explicit answer on rate-limit / window-cap scoping: failed-unlock rate limiter is **per source IP** (was global — confirmed DoS vector; the 150s stress test data shows the `operator/lifecycle/interleaver` profiles WERE blocked by the `attacker` profile's cooldown — the bug was manifesting in our own evidence); USD rolling cap remains global (signer = single writer identity, no per-IP rotation possible); (4) § 7.3.1 — explicit confirmation that the price feed URL/source is **fixed at boot env var**, NOT a parameter in the `sign` RPC (same tamper-resistance pattern as the allowlist); (5) new § 13 **Operator Runbook** with step-by-step procedures for allowlist edits, cap changes, and chain expansion.
- **v19.3-design-draft-4**: operator's final sign-off round. § 9.4 — confirmed clean socket disconnect releases the lease IMMEDIATELY (TTL is backstop for unclean crashes only); audit log distinguishes `writer_lease_released` (clean) from `writer_lease_expired` (unclean). § 7.3.4 — added **anti-truncation checkpoint** (monotonic `seq` counter + periodic checkpoint to separate file + verify script cross-check) — closes the tail-truncation gap the operator identified, stays Phase 1. § 7.3.7 — rate limiter fix **SHIPPED as v19.3.1 HOTFIX** (independent of Phase 1): new `src/lib/trading/proxy-trust.ts` with shared-secret trust model (resolves sourceIp trust issue), per-IP `Map` with TTL eviction + LRU cap (resolves unbounded growth), global aggregate cap 50 failures/1h → 1min cooldown (resolves distributed-attack gap). 3 new regression tests in `scripts/test-vault.ts`, all passing (18/18 total). The hotfix is live in production NOW; Phase 1 signer isolation can proceed independently.
- **v19.3-design-draft-5** (this revision): operator caught a critical flaw in the v19.3.1 hotfix — the `"direct-untrusted"` fallback sentinel was a single shared bucket, recreating the original self-DoS bug for the actual deployment topology (bind `127.0.0.1`, no reverse proxy since v18). Every real connection to the vault fell into the shared sentinel bucket — the hotfix did not fix the bug for the deployment running today, only for a hypothetical behind-proxy future. **v19.3.2 HOTFIX-FIX** resolves this: the fallback is now the actual TCP socket peer address (`req.socket.remoteAddress`). Implementation note: Next.js 16 App Router does NOT expose the TCP peer address to route handlers (`connection()` from `next/headers` returns `Promise<void>` — the `peer.address` API from 15.3 was removed in 16; `NextRequest.ip` was also removed). The honest equivalent: `src/instrumentation.ts` installs a monkey-patch on `http.Server.prototype.emit` (`src/lib/request-peer-capture.ts`) that intercepts the 'request' event BEFORE Next.js wraps the Node `IncomingMessage` into a Web `Request`, reads `req.socket.remoteAddress`, and stores it in an AsyncLocalStorage (`src/lib/request-peer-als.ts`). Route handlers read it via `getRequestPeerAddress()`. Non-spoofable for direct connect-

ions; gives real per-IP isolation for the deployment running today. The `"direct-untrusted"` sentinel is GONE. Other v19.3.2 changes: (a) split `proxy-trust.ts` into a pure `resolveTrustedClientIp` decision function (testable without a request context) + async `extractTrustedClientIp` production wrapper that reads from the ALS; (b) added **distinct notification event** `vault_rate_limited_global` for the global aggregate cooldown firing (severity-different signal — suggests distributed attack, not single-user brute force); (c) acknowledged the **LRU eviction trade-off** (attacker with enough real source IPs could in theory evict their own failure record — mitigated by active-cooldown entries never being evicted + the global cap being a single array independent of the per-IP Map); (d) added § 13.7 runbook entry documenting the **NAT / Docker userland-proxy edge case** where `peer.address` collapses to a single gateway IP; (e) made the test suite **self-contained** — `hardCleanupBeforeSuite()` truncates `WalletConnection` + `ExchangeConnection` tables at suite start, so the suite no longer depends on the operator manually cleaning wallets between runs (the "test fails → clean DB manually → rerun → passes" pattern that happened 3 times in this thread is resolved at the suite level). 2 new regression tests: scenario 18 (two distinct direct IPs do NOT share a bucket — THE test the operator required) + scenario 19 (forged XFF without proxy secret is ignored, peer address is used). Suite now 20/20 passing.

- **v19.3-design-draft-6** (this revision): operator review of draft-5 caught a critical pattern — the assertion "v19.3.2 SHIPPED, Phase 1 ready without reservations" was being treated as fact without the operator having seen the evidence. The operator's review identified three remaining issues in § 7.3.7: **(1) NO try/catch on the monkey-patch path** — the `patchedEmit` function in `request-peer-capture.ts` did not wrap the peer-address capture nor the `runWithPeerAddress` call in try/catch. A single exception in either would propagate up through `http.Server.prototype.emit` and crash the server on EVERY request — a strictly worse failure mode than the silent-crash bug this whole investigation is about, and exactly the class of bug that the investigation has been trying to understand. The operator's directive: any error in the capture must degrade to the shared `"unidentifiable-socket"` bucket, NEVER propagate the exception through `emit()`. **(2) Tests did NOT exercise the riskiest component** — scenarios 18 and 19 in `test-vault.ts` called the PURE decision function `resolveTrustedClientIp({ peerAddress, ... })` with synthetic peer-address values. They validated the decision logic downstream of the capture, but did NOT exercise the actual `http.Server.prototype.emit` monkey-patch nor the AsyncLocalStorage propagation under concurrent requests. This is the same pattern that bit this thread three times already (vault-empty test always "succeeds", CHECK 2 false-negative by timing): the riskiest piece of code is hidden behind a test that only validates the downstream logic. **(3) No documentation of the http.Server coupling** — the capture mechanism depends on requests flowing through `http.Server.prototype.emit('request', req, res)`. A future migration to HTTP/2 (which emits 'stream' instead of 'request'), a custom server adapter, or a serverless platform that synthesizes `req` objects would silently degrade the rate limiter to the shared `"unidentifiable-socket"` bucket — back to the v19.3.1 self-DoS shape, just with a different bucket name. Draft-6 addresses all three: (a) `request-peer-capture.ts` now wraps both the capture and the ALS dispatch in try/catch with documented fallback (`peerAddress=null` → named bucket, request still completes, warning written to stderr); (b) new `scripts/test-request-peer-integration.ts` exercises the

REAL monkey-patch + ALS with a live `http.Server` and two distinct loopback source IPs (`127.0.0.1` + `127.0.0.2`, both in `127.0.0.0/8` per RFC 5735) under 10 concurrent requests-in-flight, plus an exception-path test that emits a fake 'request' event with a Proxy-throws `req.socket` and confirms the server does NOT crash; (c) explicit COUPLING-TO-`http.Server` comment block in `request-peer-capture.ts` documents the silent-degradation cases (HTTP/2, custom server, serverless adapter) with detection query and migration path. Also cleaned up § 11 — the obsolete "wallet-crypto.ts per-IP rate limit refactor — 0.5 day" line is removed from Phase 1 estimates (the work shipped independently as the v19.3.1/v19.3.2 hotfix). **Phase 1 remains BLOCKED on operator re-review of § 7.3.7.**

0. Executive Summary

This document proposes moving the wallet vault (decrypted private keys + signing operations) out of the Next.js web process into a **dedicated child process** that communicates with the web process via a **Unix domain socket** using a **JSON-RPC 2.0** protocol with a strict method allowlist.

The goal is **blast-radius containment**: a compromise of the web process (SSRF, prototype pollution, malicious npm dependency, the silent-crash bug itself) must not grant immediate access to decrypted private keys. The keys live in a separate process with a separate address space, and the only way to use them is through an audited RPC interface that exposes `sign()` but not `exportKey()`.

This document is the design only. Implementation is a separate work item. The operator explicitly directed: advance the design now, run the two crash checks in parallel, report both together.

v19.3 operator review update (BLOCKING): the operator's review of this draft identified a critical gap in the original design. Process isolation alone protects against a compromised web process *reading* keys, but does NOT protect against a compromised web process *using* the signer as a signing oracle through the legitimate IPC channel. If the signer blindly trusts any `sign` RPC that arrives on the socket, and the `hotWalletCapUsd` check lives only in `portfolio.ts` (web side), then an attacker who achieves RCE in the web process via a supply-chain dependency (the original motivating threat) does not need to steal the key — they just send a valid `sign` instruction for an arbitrarily large transaction to an arbitrary address. Memory isolation without **signer-side authorization policy** is containment of one failure type, not of the one that motivated the isolation.

This revision makes the following changes (all marked **v19.3**):

- **§ 7.3 added**: Signer-Side Authorization Policy — per-transaction value cap, per-time-window rolling cap, destination address allowlist, and an

independent signer-written audit log. This is the change that transforms "process isolation" into a real "trust boundary".

- **§ 8 Phase 1 reordered:** the authorization policy is now a Phase 1 BLOCKING prerequisite, not a Phase 3 future consideration. The original Phase 3 "signer-side transaction validation" item is split — policy enforcement moves to Phase 1; higher-level intent-based signing stays in Phase 3 (it requires the signer to understand trading semantics).
- **§ 9 answers tightened** to the operator's stated defaults (stricter than the original recommendations): passphrase never crosses IPC, restart is fail-closed, socket parent dir checked for symlink swap, single-writer enforced, signer deps minimal + pinned.
- **§ 12.2 added:** clean 150s heterogeneous stress test result (CHECK 1 negative, 247 cycles, zero organic crashes) + port-collision hypothesis investigation findings (INDETERMINATE — no direct EADDRINUSE evidence, but hypothesis structurally plausible and consistent with "doesn't reproduce in production").

1. Threat Model

1.1 What we are defending against

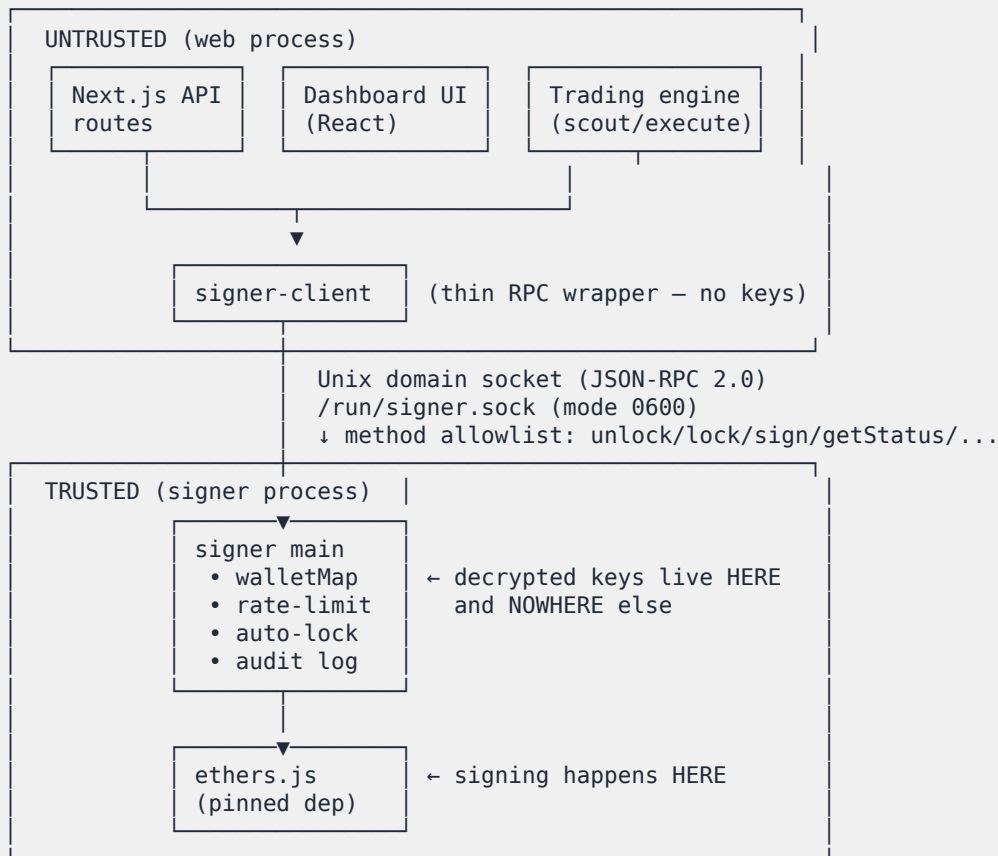
Threat	How process isolation helps
Malicious npm dependency in web process	A backdoored package imported by the web app cannot call <code>walletVault.getWalletKey()</code> directly — that method no longer exists in the web process. It can only send RPCs to the signer, which only allows <code>sign()</code> + <code>getStatus()</code> , not <code>exportKey()</code> .
SSRF / prototype pollution achieving RCE in web process	Even with full code execution in the web process, the attacker cannot dump keys from memory — the keys are not there. They can send forged RPCs, but the method allowlist limits what they can do. (Request signing in Phase 3 raises the bar further.)
Memory inspection of web process	A <code>process.memoryUsage()</code> dump or heap snapshot of the web process yields zero private keys. The attacker would need to inspect the signer process separately.
Web process crash leaving keys in memory	The current design holds decrypted keys in the web process's heap. If it crashes (the silent-crash bug), the keys may persist in a core dump or swap. With isolation, the signer detects the web process dying (socket close) and immediately zeros all keys + exits , so no keys survive.
Supply-chain attack on signing library	If <code>ethers.js</code> or a signing dependency has a vulnerability, the blast radius is limited to the signer process. The web process is unaffected, and the operator can pin/audit the signer's dependencies independently.

1.2 What process isolation does NOT defend against (honest limitations)

Non-threat	Why not
------------	---------

Local attacker with debugger (ptrace, gdb)	An attacker with CAP_SYS_PTRACE or root can attach to the signer process and read its memory. Process isolation doesn't help against local code execution. This is the same limitation as the current design — we're not making it worse.
Kernel compromise	A kernel-level attacker can read any process's memory via /proc/<pid>/mem or similar. Out of scope.
Disk theft (offline brute-force of encrypted blobs)	The encrypted blobs are still on disk. This is addressed by the Argon2id KDF migration (v19 item 3), not by process isolation.
Signer process itself compromised	If the signer has a vulnerability (e.g., a bug in ethers.js transaction signing), the keys are still at risk. Process isolation limits the blast radius of web-process compromises, not signer-process compromises. The signer must still be audited.
Passphrase interception at the IPC boundary	The passphrase crosses from web to signer via RPC. A compromised web process can intercept it before it reaches the signer. v19.3: § 9.1 changed — the operator's default is that the passphrase never crosses IPC at all (signer owns the input lifecycle). See § 9.1 for the revised recommendation.
Compromised web process forging sign RPCs	v19.3 — this row previously listed as a non-threat; corrected. A compromised web process can send arbitrary sign RPCs through the legitimate IPC channel. The original draft of this document hand-waved this away as "the defense is the trading engine's risk checks, which run in the web process" — but if the web process is compromised, those checks are bypassed. This is exactly the threat the operator's v19.3 review called out as the most important one. Mitigation: § 7.3 (Signer-Side Authorization Policy) — the signer enforces its own per-tx cap, per-window cap, destination allowlist, and independent audit log. The web process's risk checks become defense-in-depth, not the primary boundary.

1.3 Trust boundaries



The **IPC socket is the only trust boundary crossing**. Everything north of it is untrusted (assumed potentially compromised); everything south of it is trusted (audited, pinned dependencies, minimal surface).

2. Architecture

2.1 Process model

```

Parent: Next.js (next start)
├─ instrumentation.ts → spawnSigner()
│   └─ child_process.spawn("node", ["dist/signer/main.js"], { stdio: ... })
├─ signer-client.ts → connects to /run/signer.sock
├─ API routes (/api/vault, /api/wallets) → call signer-client
└─ trading engine → calls signer-client.sign() for live txs

Child: signer (node dist/signer/main.js)
├─ creates /run/signer.sock (mode 0600, owner = web process user)
├─ listens for JSON-RPC 2.0 over the socket
├─ holds walletVault (moved from wallet-crypto.ts)
├─ on parent disconnect (socket close) → zero keys + exit
└─ on uncaughtException → zero keys + crash log + exit(1)
  
```

2.2 Why Unix domain socket (not stdin/stdout, not TCP, not named pipe)

Option	Verdict	Reasoning
Unix domain socket (AF_UNIX)	✓ Chosen	Local-only (no network exposure), filesystem permissions (0600) provide authentication, supports concurrent connections, Node.js <code>net</code> module has first-class support, standard for local IPC.
stdin/stdout JSON-RPC (LSP-style)	✗ Rejected	Simpler, but stdin is a single stream — hard to support concurrent requests cleanly. Also couples the signer's logging (which wants stdout) with the protocol.
TCP localhost	✗ Rejected	Exposes the signer to the network stack (even on 127.0.0.1, a SSRF in the web process could reach it). No filesystem permission model.
Named pipe (Windows)	✗ N/A	Not targeting Windows.

2.3 Why JSON-RPC 2.0 (not custom binary, not gRPC)

Option	Verdict	Reasoning
JSON-RPC 2.0	✓ Chosen	Human-readable (easy to audit in crash logs), standard library support, supports request/response + notifications, no code generation step.
Custom binary protocol	✗ Rejected	Slightly faster, but opaque in logs. For the volume we're doing (single-digit RPCs per trade), JSON overhead is negligible.
gRPC	✗ Rejected	Requires protobuf codegen + native deps. Overkill for a 2-process local system.

3. IPC Protocol

3.1 Transport

- **Socket path:** `/run/signer.sock` (configurable via `SIGNER_SOCKET_PATH`)
- **Fallback:** `${XDG_RUNTIME_DIR}/signer.sock` or `/tmp/signer-<uid>.sock`
- **Permissions:** `0600` (owner read/write only), owner = the user running the web process
- **Connection model:** the web process opens ONE persistent connection at boot and reuses it for all RPCs. If the connection drops, the client treats the vault as locked and attempts reconnect.
- **Framing:** newline-delimited JSON (each JSON-RPC message is one line). Simple, robust, easy to debug.

3.2 Message format (JSON-RPC 2.0)

Request (web → signer):

```
{"jsonrpc": "2.0", "id": "<uuid>", "method": "unlock", "params": {"passphrase": "...", "sourceIp": "203.0.113.42"}}
```

Response (signer → web):

```
{"jsonrpc": "2.0", "id": "<uuid>", "result": {"wallets": 5, "exchanges": 0}}
```

Error response:

```
{"jsonrpc": "2.0", "id": "<uuid>", "error": {"code": -32001, "message": "Passphrase incorreta", "data": {"recentFailures": 3}}}
```

Notification (signer → web, no response expected):

```
{"jsonrpc": "2.0", "method": "vault_auto_locked", "params": {"reason": "30min idle"}}
```

3.3 Method allowlist

Methods the signer accepts:

Method	Params	Result	Notes
health_check	{}	{ ok: true, pid: number, uptime: number, vaultUnlocked: boolean }	No side effects. Used at boot + periodic keepalive.
unlock	{ passphrase: string, sourceIp: string }	{ wallets: number, exchanges: number }	Loads + decrypts all blobs. Rate-limited (5 failures/60s → 5min cooldown).
lock	{ reason: string, sourceIp: string }	{ ok: true, walletCount: number, exchangeCount: number }	Wipes all keys.

<code>get_status</code>	<code>{}</code>	<code>{ unlocked, walletCount, exchangeCount, unlockTime, autoLockInSec, cooldownUntil, recentFailures }</code>	Read-only. Used by dashboard.
<code>get_public_info</code>	<code>{ walletId: string }</code>	<code>{ label, address, type, chain }</code>	Returns ONLY public metadata. Never the private key.
<code>sign</code>	<code>{ walletId: string, txData: object, sourceIp: string }</code>	<code>{ signedTx: string, txHash: string }</code>	The critical operation. Signs a transaction with the decrypted key. The key never leaves the signer. v19.3: before signing, the signer enforces the authorization policy from § 7.3 — per-tx value cap, per-window rolling cap, destination allowlist, independent audit log. A sign RPC that violates the policy is rejected with -32006 Policy violation (see § 3.4) and logged to the signer's independent audit log. The web process cannot bypass or configure this policy at runtime — it is loaded from env vars at signer boot.
<code>sign_typed_data</code>	<code>{ walletId: string, domain, types, message, sourceIp: string }</code>	<code>{ signature: string }</code>	EIP-712 typed data signing (for DEX approvals, permit signatures). v19.3: same authorization policy as sign applies — the domain.verifyContract is checked against the destination allowlist, and the permit value is checked against the per-tx + per-window caps.
<code>create_wallet</code>	<code>{ label, type, address, privateKey, passphrase, sourceIp }</code>	<code>{ id: string }</code>	Creates a wallet row + encrypts the key. The plaintext key is seen only by the signer.
<code>delete_wallet</code>	<code>{ id: string, sourceIp: string }</code>	<code>{ ok: true }</code>	Deletes a wallet row. If the vault is unlocked, also wipes the in-memory key.
<code>shutdown</code>	<code>{ reason: string }</code>	<code>{ ok: true }</code>	Polite shutdown: zero keys, close socket, exit 0. Sent by the web process on SIGINT/SIGTERM.

Methods that MUST NEVER be exposed (the security-critical allowlist boundary):

Forbidden method	Why
<code>export_key</code>	Private keys must never leave the signer process.
<code>get_wallet_key</code>	Direct key access stays in-process.
<code>raw_decrypt</code>	The signer's decryption capability is for vault unlock only, not arbitrary blobs.
<code>eval</code> / <code>execute</code> / <code>run_script</code>	No arbitrary code execution. The signer is a signing oracle, not a general-purpose worker.
<code>read_file</code> / <code>write_file</code>	No filesystem access through the RPC interface.

The allowlist is enforced as a hard-coded `Set<string>` in the signer's request dispatcher. Any method not in the set returns a `-32601 Method not found` error. There is no way to add methods at runtime.

3.4 Error codes

Code	Meaning
<code>-32700</code>	Parse error (malformed JSON)
<code>-32600</code>	Invalid request (not valid JSON-RPC 2.0)
<code>-32601</code>	Method not found (not in allowlist)
<code>-32602</code>	Invalid params (missing required field)
<code>-32603</code>	Internal error (uncaught exception in the handler — signer will also write a crash log)
<code>-32001</code>	Vault locked (unlock required first)
<code>-32002</code>	Rate limited (cooldown active)
<code>-32003</code>	Wallet not found
<code>-32004</code>	Signing failed (ethers.js error)
<code>-32005</code>	Passphrase incorrect (unlock failed)
<code>-32006</code>	v19.3: Policy violation — <code>sign</code> or <code>sign_typed_data</code> RPC rejected by the signer-side authorization policy (§ 7.3). The rejection is logged to the signer's independent audit log + fires a notification. The web process cannot suppress this.

4. Process Lifecycle

4.1 Boot sequence

Next.js process boots (instrumentation.ts)

- 1. registerCrashHandlers() (existing – v19-BLOCKING)
- 2. spawnSigner()
 - child_process.spawn("node", ["dist/signer/main.js"], {
 - stdio: ["pipe", "pipe", "pipe"],
 - env: { SIGNER_SOCKET_PATH: "/run/signer.sock", ... }
 - })
 - wait for child stderr to print "SIGNER_READY socket=/run/signer.sock" (max 10s timeout – if not ready, abort boot)
 - store child PID for liveness monitoring
- 3. signerClient.connect("/run/signer.sock")
 - open persistent connection
- 4. health_check RPC → expect { ok: true, vaultUnlocked: false } (if this fails, abort boot – signer is unhealthy)
- 5. Continue with normal Next.js boot (route handlers, etc.)

4.2 Shutdown sequence (graceful)

Next.js receives SIGINT or SIGTERM

- 1. signerClient.shutdown({ reason: "SIGTERM" })
 - signer receives shutdown RPC
 - signer zeros all keys (overwrite + clear Maps)
 - signer writes "vault_locked shutdown" to audit log
 - signer closes the socket
 - signer exits 0
- 2. wait for child process to exit (max 5s)
- 3. process.exit(130) (existing SIGINT handler in crash-logger.ts)

4.3 Failure: signer crash

Signer process dies (uncaughtException → crash handler → exit 1)

- Web process detects: socket receives 'close' event
- signerClient.onDisconnect handler fires:
 - mark vault as LOCKED in the cached status (conservative)
 - log error: "Signer process disconnected – vault assumed locked"
 - notify operator: "Signer crash – vault locked, trading halted"
 - schedule reconnect attempt (exponential backoff: 1s, 2s, 4s, 8s, max 30s)
- Reconnect attempts:
 - try to spawn a new signer process
 - if spawn succeeds + health_check passes → vault is in LOCKED state (keys were zeroed on crash – operator must re-unlock)
 - if spawn fails 3 times → give up, web process continues in degraded mode (all vault operations return 503, trading engine halts)
 - operator must manually investigate + restart
- Crash log: the signer wrote logs/crash-*.log BEFORE exiting (same sync mechanism as the web process – the signer also uses crash-logger.ts)

4.4 Failure: web process crash

```
Next.js process dies (the silent-crash bug, or OOM, or operator kill)
├─ Signer process detects: socket receives 'close' event on the accepted connection
├─ Signer's disconnect handler fires:
│   ├── THIS IS THE KEY SECURITY PROPERTY
│   ├── zero all keys immediately (overwrite + clear Maps)
│   ├── write "vault_locked parent_disconnect" to audit log
│   ├── write crash-log-style entry: "Parent process disconnected, zeroing keys"
│   └─ exit 0 (clean exit – the signer did its job)
└─ Result: no decrypted keys survive the web process crash.
    The encrypted blobs are still on disk (unchanged), but the in-memory
    decrypted copies are gone. An attacker who inspects the machine after
    the crash finds zero keys in memory.
```

4.5 Failure: IPC timeout

```
Web process sends RPC → no response within 5s (configurable)
├─ signerClient throws TimeoutError
├─ Caller (API route or trading engine) receives the error:
│   ├── API route → returns 503 "Signer timeout – vault unavailable"
│   └─ Trading engine → treats as "vault locked", halts EXECUTE phase
├─ The signer may still be processing (e.g., long Argon2id derivation
    could take >5s on weak hardware). When it finishes, it sends the
    response, which the web client ignores (correlation ID no longer
    in the pending map).
└─ If timeouts persist across multiple RPCs, the web process initiates
    the signer-crash recovery sequence (§4.3).
```

4.6 Failure: partial failure (signer alive but vault locked)

```
Signer is running but vault is locked (e.g., auto-lock fired, or signer
restarted after crash)
├─ get_status → { unlocked: false }
├─ sign RPC → error -32001 "Vault locked"
├─ Trading engine sees vault locked:
│   ├── halts EXECUTE phase (cannot sign transactions)
│   ├── continues MONITOR phase (can still read positions from DB)
│   └─ notifies operator: "Vault locked – trading paused, re-unlock required"
└─ Operator re-enters passphrase via /api/vault → unlock RPC → signer
    decrypts → trading resumes.
```

5. State Ownership

State	Owner	Rationale
Decrypted private keys	Signer only	The entire point of isolation. Never serialized, never sent over IPC, never in web process memory.
Passphrase (transient)	Signer only	Enters via unlock RPC, held in a local variable during decryption, then overwritten with zeros. Never persisted, never logged.
Encrypted blobs	DB (shared)	Both processes can read the DB, but only the signer decrypts. The web process reads only public columns (label, address, type).
Wallet metadata (label, address, type)	DB (shared)	Web process reads for dashboard display. Signer reads during unlock to know which rows to decrypt.
Vault lock/unlock state	Signer (source of truth)	Web process maintains a cached copy for UI, but the signer is authoritative. If they disagree, the signer wins.
Rate-limit counters (failedAttempts, cooldownUntil)	Signer only	Anti-brute-force state must live in the signer — otherwise a compromised web process could just reset the counters.
Auto-lock timer	Signer only	The signer tracks lastKeyAccessAt and fires auto-lock independently. The web process cannot disable it.
v19.3: Authorization policy (caps + allowlist)	Signer only	The per-tx cap, per-window rolling cap, and destination allowlist are loaded from env vars at signer boot (SIGNER_MAX_TX_USD , SIGNER_MAX_WINDOW_USD , SIGNER_WINDOW_SEC , SIGNER_DEST_ALLOWLIST). The web process cannot read or modify these at runtime — they live in the signer's process memory.
v19.3: Rolling window spend tracker	Signer only	The signer maintains a rolling window of signed transaction values (used to enforce the per-window cap). A compromised web process cannot reset this counter to bypass the cap.
v19.3: Independent signer audit log	Signer only (write); operator (read)	The signer writes its own audit log to logs/signer-audit.jsonl (append-only, JSON Lines). Each entry is a signed transaction or a policy rejection. The web process CANNOT write to this file — it has no filesystem path to it (the path is configurable only via signer env var). This is the "trust boundary" the operator asked for: if the web process is compromised, it can lie in its own AppLog rows, but it cannot rewrite what the signer already recorded about what it signed.
Audit log (AppLog)	DB (shared)	Both processes write to it. The signer writes vault events; the web process writes API + engine events. v19.3: the signer's independent audit log (above) is the tamper-evident record; AppLog is now the secondary, web-process-writable log.
Crash logs	Separate per process	Web: logs/crash-*.log . Signer: logs/signer-crash-*.log . Keeps them distinct for post-mortem.

6. Concurrency Model

6.1 Signer is single-threaded

The signer is a Node.js process — single event loop, no true parallelism. RPC requests are processed sequentially. This is **intentional**:

- **No concurrent key access:** two `sign()` calls for the same wallet cannot interleave and produce corrupted signatures. The signer processes them one at a time.
- **No TOCTOU on vault state:** the `unlock` → `decrypt all blobs` → `populate Maps` sequence is atomic from the perspective of other RPCs. A `sign()` RPC that arrives during unlock either sees the old state (locked) or the new state (unlocked), never a partial state.
- **Simpler reasoning:** the current `wallet-crypto.ts` already relies on single-threaded semantics for its Map mutations. Moving to a separate process preserves this invariant.

6.2 Request queueing

The web process may send multiple RPCs concurrently (e.g., the dashboard polls `get_status` while the trading engine calls `sign`). The signer queues them in arrival order and processes sequentially. Each response carries the correlation ID from the request, so the web client matches responses to requests correctly even under concurrency.

6.3 Long operations

`unlock` does PBKDF2 (600k iterations) × N wallets. With 5 wallets, this is ~2-3 seconds on typical hardware. During this time, the signer's event loop is blocked (PBKDF2 is synchronous in Node's `crypto` module). Other RPCs queue behind it.

This is acceptable because:

- `unlock` is infrequent (once at engine start, then after auto-lock)
- The web process shows a "Unlocking..." spinner during the RPC
- The 5s IPC timeout may need to be extended to 15s for `unlock` specifically

When we migrate to Argon2id (v19 item 3), the derivation time increases (~5s per attempt on default params). The `unlock` RPC timeout should be 30s to accommodate this.

7. Security Properties

7.1 Guarantees provided

1. **No decrypted key in web process memory, ever.** The web process never imports `wallet-crypto.ts`'s `walletVault`. It imports `signer-client.ts`, which holds only a socket connection + a cached status snapshot.
2. **Key zeroing on web process death.** The signer detects socket close and zeros keys within milliseconds. No core dump of the web process contains keys (they were never there).
3. **Method allowlist is the attack surface.** A compromised web process can only invoke the 10 methods in § 3.3. It cannot call `export_key` (doesn't exist), cannot call `eval` (doesn't exist), cannot read arbitrary files.
4. **Rate limiting is signer-enforced.** A compromised web process cannot bypass the brute-force rate limit by directly manipulating the `failedAttempts` array — that array lives in the signer.
5. **Auto-lock is signer-enforced.** A compromised web process cannot disable the 30-min idle auto-lock by resetting `lastKeyAccessAt` — that timestamp lives in the signer.
6. **Audit log is signer-written.** Vault events (unlock, lock, sign) are written by the signer directly to the DB. A compromised web process cannot suppress them (though it could write its own fake events — see § 7.2).

7.2 Non-guarantees (what a compromised web process CAN do)

1. **Forge unlock RPCs.** A compromised web process can send `unlock` with a passphrase it intercepted from the operator's UI input. This is the passphrase-interception limitation in § 1.2. Mitigation: Phase 3 request signing (HMAC with a shared secret established at boot), or direct passphrase entry in the signer (§ 9.1).
2. **Forge sign RPCs.** A compromised web process can ask the signer to sign arbitrary transactions. **v19.3 — this non-guarantee is now PARTIALLY closed by § 7.3 (Signer-Side Authorization Policy).** The signer enforces:
 - Per-transaction value cap (`SIGNER_MAX_TX_USD`, default = `hotWalletCapUsd` from config) — a sign RPC for a 10,000tx is rejected when the cap is 200.
 - Per-time-window rolling cap (`SIGNER_MAX_WINDOW_USD` over `SIGNER_WINDOW_SEC`, default \$500 / 300s) — even if each individual tx is under the per-tx cap, the signer rejects once the rolling window sum exceeds the per-window cap.
 - Destination address allowlist (`SIGNER_DEST_ALLOWLIST`, a JSON file path loaded at boot) — only known DEX routers / spender contracts are accepted. A sign RPC to an arbitrary EOA is rejected.

- Independent audit log (`logs/signer-audit.jsonl`) — every sign RPC (accepted or rejected) is recorded by the signer, in a file the web process cannot write to or rewrite.

What this STILL does not prevent: a compromised web process sending a sign RPC for a *policy-compliant* transaction (e.g., a \$150 swap to a known DEX router) that the operator did not authorize. Full mitigation requires Phase 3 intent-based signing (the signer takes a higher-level intent like "close position X" and constructs the tx itself, so the web process cannot forge the destination or amount). The operator should treat § 7.3 as raising the bar from "any signed tx" to "a policy-compliant signed tx" — not as fully closing the gap.

3. **Suppress notifications.** A compromised web process could swallow the `vault_unlocked` notification before it reaches the notifier. Mitigation: the signer should also write directly to the notifier (Phase 3 — signer holds its own notifier client).
4. **Write fake audit log entries.** A compromised web process can write arbitrary rows to `AppLog`. This is a limitation of the shared-DB model. Mitigation: Phase 3 could add a signer-written hash chain to AppLog entries, making forgery detectable.

7.3 Signer-Side Authorization Policy (v19.3 — BLOCKING for Phase 1)

This section is the operator's v19.3 review point #3. The original draft of this document treated the signer as a pure signing oracle — it would sign any well-formed `sign` RPC that arrived on the socket. That is process isolation without a trust boundary: it protects against the web process *reading* keys, but not against the web process *using* the signer as a drain. The operator's exact framing:

*"Isolamento de memória sem **política de autorização própria dentro do signer** é contenção de um tipo de falha, não de todos os que motivaram o isolamento. ... Eu exigiria, antes de aprovar a Fase 1 de implementação: o signer precisa reimplementar e reforçar, do lado dele, pelo menos [teto de valor por transação e por janela de tempo, allowlist de endereços/contratos de destino, log de auditoria próprio do signer]."*

This section specifies those three requirements plus the configuration model that makes them tamper-resistant against a compromised web process.

7.3.1 Per-transaction value cap

Rule: the signer rejects any `sign` RPC whose transaction value (in USD) exceeds `SIGNER_MAX_TX_USD` (env var, default = the same `hotWalletCapUsd` value from `src/lib/trading/config.ts` — currently \$200).

How the value is computed:

- For a simple ETH transfer: `tx.value` × spot ETH/USD (fetched via the signer's own price feed — the signer does NOT trust the web process's price).
- For an ERC-20 transfer (decoded from `tx.data` against the destination contract, which must be in the allowlist per § 7.3.3): the decoded `amount` × the token's spot USD price.
- For a DEX swap (decoded from `tx.data` against a known router): the input token amount × spot USD price. The output token is NOT counted (avoid double-counting with the per-window cap).
- If the signer cannot decode the tx (unknown contract, unknown method selector): **reject** with `-32006 Policy violation (undecodable tx)`. Fail-closed is the only safe default.

Why the signer does its own price fetch: a compromised web process could lie about the USD value of a transaction to bypass the cap. The signer must fetch the price independently (from the same sources the web process uses — Binance + DexScreener — but via its own HTTP client, its own cache, its own rate-limit handling). If the price fetch fails, the signer **rejects** the sign RPC with `-32006 Policy violation (price unavailable)`. Fail-closed.

Price feed source is fixed at boot — NOT a parameter of the `sign` RPC (v19.3 operator review point #4). The URL(s) and source list (Binance REST endpoints, DexScreener base URL) are read from the signer's env vars at boot:

Env var	Default	Purpose
<code>SIGNER_PRICE_FEED_BINANCE_URL</code>	<code>https://api.binance.com/api/v3/ticker/price</code>	Binance spot price endpoint.
<code>SIGNER_PRICE_FEED_DEXSCREENER_URL</code>	<code>https://api.dexscreener.com/latest/dex/tokens</code>	DexScreener token price endpoint.
<code>SIGNER_PRICE_FEED_STALE_SEC</code>	<code>60</code>	Max age of a cached price before sign RPC is rejected as <code>price unavailable</code> .

The `sign` RPC payload schema (§ 3.3) does NOT include a `priceFeedUrl` or `price` field. A web process that tries to pass one is rejected with `-32602 Invalid params (unexpected field: priceFeedUrl)`. This is the same

tamper-resistance pattern as the destination allowlist (§ 7.3.3) and the value caps (§ 7.3.5): the policy inputs are bound at boot, and the only way to change them is to edit the env var and restart the signer — which, per § 9.2, requires manual re-unlock. A compromised web process cannot manipulate the price used to validate the cap, because the web process never gets to specify the price.

Configuration: `SIGNER_MAX_TX_USD` is read from env at signer boot. The web process cannot change it at runtime (no RPC method exists to update the cap; the signer has no `set_policy` method in its allowlist). To change the cap, the operator edits the env var and restarts the signer (which, per § 9.2, requires a manual re-unlock).

7.3.2 Per-time-window rolling cap

Rule: the signer tracks the sum of signed transaction values over a rolling window of `SIGNER_WINDOW_SEC` (default 300s / 5min). If a new sign RPC would push the rolling sum above `SIGNER_MAX_WINDOW_USD` (default \$500), the signer rejects it with `-32006 Policy violation (window cap exceeded)`.

Why this matters: without the per-window cap, a compromised web process could drain the hot wallet by sending many small, individually-policy-compliant transactions (each \$199, all to the same allowlisted router) until the wallet is empty. The per-window cap bounds the total bleed rate.

State: the rolling window is an in-memory array of `{ timestamp: number, valueUsd: number }` entries, pruned to the last `SIGNER_WINDOW_SEC` seconds on each sign RPC. The array lives in the signer process — a compromised web process cannot read it, cannot reset it, cannot fudge the timestamps. If the signer restarts (crash or operator restart), the window is wiped — this is conservative in the right direction (the cap is *more* restrictive after a restart, not less, because the rolling sum starts at zero).

Scope: GLOBAL (per signer), not per source identity (v19.3 operator review point #3). This is the correct scope for the USD cap because the signer is **single-writer** (§ 9.4): only one connection (the web process) can hold the writer lease at a time, and only the writer can call `sign`. There is no "per-IP" rotation attack at the signer layer — the socket is mode `0600` (§ 9.3), so only the web process can connect at all. Per-IP scoping of the USD cap would weaken the cap without adding any defense: an attacker who has compromised the web process IS the web process, and the cap exists precisely to bound what that single compromised identity can drain. Keeping it global means the cap bounds the bleed rate regardless of how the web process tries to subdivide its requests.

Contrast with the failed-unlock rate limiter (§ 7.3.7): the rate limiter on failed `unlock` attempts is per source IP, because unlock attempts arrive via the HTTP API (not the writer-locked IPC channel) and CAN come from multiple identities concurrently. See § 7.3.7 for the scoping analysis and the bug it uncovered in the 150s stress test data.

Failure mode: if the rolling sum cannot be computed (e.g., a previous entry's USD value is unknown because the price fetch had failed at sign time — but that case is already rejected per § 7.3.1, so this shouldn't happen), the signer rejects the new RPC with `-32006 Policy violation (window state corrupt)`.

7.3.3 Destination address allowlist

Rule: the signer rejects any `sign` RPC whose `tx.to` address is not in the loaded allowlist. The allowlist is loaded at boot from `SIGNER_DEST_ALLOWLIST` (env var, path to a JSON file). Example file:

```
{
  "version": 1,
  "loadedAt": "2026-07-14T10:00:00Z",
  "entries": [
    {
      "address": "0xE592427A0AEce92De3Edee1F18E0157C05861564",
      "label": "Uniswap V3 Router (Ethereum mainnet)",
      "chainId": 1,
      "methods": ["exactInputSingle", "exactInput", "exactOutputSingle", "exactOutput"]
    },
    {
      "address": "0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45",
      "label": "Uniswap V3 Router 2 (with swaps)",
      "chainId": 1,
      "methods": ["*"]
    },
    {
      "address": "0xdef1c0ded9bec7f1a1670819833240f027b25EfF",
      "label": "0x ExchangeProxy (permit2-based)",
      "chainId": 1,
      "methods": ["*"]
    }
  ]
}
```

For `sign_typed_data` (EIP-712): the `domain.verifyingContract` is checked against the same allowlist. A permit signature for an unknown contract is rejected.

Why an allowlist and not a blocklist: a blocklist (blocked addresses) would require the signer to know every malicious address — impossible. The allowlist is fail-closed: only known-good destinations are accepted. Adding a new destination requires the operator to edit the JSON file and restart the signer (which requires re-unlock per § 9.2). This is intentional friction — the operator should think carefully before adding a new destination.

Configuration: `SIGNER_DEST_ALLOWLIST` is a path (default `/etc/signer/dest-allowlist.json`). The file is read at boot; if it doesn't exist or is malformed, the signer refuses to start (fail-closed). The web process has no RPC method to add/remove entries at runtime.

7.3.4 Independent signer audit log

Rule: the signer writes its own append-only audit log to `SIGNER_AUDIT_LOG` (env var, default `logs/signer-audit.jsonl`). Every `sign` and `sign_typed_data` RPC — whether accepted or rejected — produces one JSON-Lines entry. Example entries:

```
{
  "ts": "2026-07-14T10:23:01.234Z",
  "rpcId": "abc-123",
  "method": "sign",
  "walletId": "w_001",
  "to": "0xE592...1564",
  "valueUsd": 150.00,
  "status": "signed",
  "txHash": "0x...",
  "rollingWindowSum": 150.00
}
{
  "ts": "2026-07-14T10:23:05.567Z",
  "rpcId": "def-456",
  "method": "sign",
  "walletId": "w_001",
  "to": "0xAttacker...",
  "valueUsd": 10000.00,
  "status": "rejected",
  "reason": "destination_not_in_allowlist",
  "rollingWindowSum": 150.00
}
{
  "ts": "2026-07-14T10:23:10.890Z",
  "rpcId": "ghi-789",
  "method": "sign_typed_data",
  "walletId": "w_001",
  "verifyingContract": "0xdef1...b25EfF",
  "valueUsd": 50.00,
  "status": "signed",
  "signature": "0x...",
  "rollingWindowSum": 200.00
}
{
  "ts": "2026-07-14T10:23:15.123Z",
  "rpcId": "jkl-012",
  "method": "sign",
  "walletId": "w_001",
  "to": "0xE592...1564",
  "valueUsd": 350.00,
  "status": "rejected",
  "reason": "window_cap_exceeded",
  "rollingWindowSum": 200.00,
  "windowCap": 500.00
}
```

Why this matters (the operator's exact point): "se o processo web for comprometido, ele pode mentir no próprio log; o signer não deveria depender dele para ter um registro confiável do que assinou." The web process can write arbitrary rows to `AppLog` (it has DB write access). It cannot write to `signer-audit.jsonl` because:

1. The web process does not know the file path (it's configurable only via the signer's env var, which the web process does not read).
2. Even if the web process discovered the path, writing to a file the signer also writes to would interleave entries — detectable on review (entries from the web process would not match the signer's `rpcId` sequence).
3. **v19.3 (Phase 1, mandatory — operator review point #2):** each entry includes `prevHash = sha256(canonicalJson(prevEntry))`, forming a hash chain from the genesis entry. Retroactive modification, deletion, or insertion of any entry breaks the chain visibly. The operator can verify chain integrity at any time with `scripts/verify-signer-audit.ts` (added to Phase 1 scope, see § 8). This is a few lines of code on the write side and a single linear scan on the verify side — cheap enough to belong in Phase 1, not Phase 2.

Hash chain schema (Phase 1):

```
{
  "ts": "2026-07-14T10:23:01.234Z",
  "rpcId": "abc-123",
  "method": "sign",
  "walletId": "w_001",
  "to": "0xE592...1564",
  "valueUsd": 150.00,
  "status": "signed",
  "txHash": "0x...",
  "rollingWindowSum": 150.00,
  "prevHash": "0000...0000",
  "entryHash": "9f3a..."
}
{
  "ts": "2026-07-14T10:23:05.567Z",
  "rpcId": "def-456",
  "method": "sign",
  "walletId": "w_001",
  "to": "0xAttacker...",
  "valueUsd": 10000.00,
  "status": "rejected",
  "reason": "destination_not_in_allowlist",
  "rollingWindowSum": 150.00,
  "prevHash": "9f3a...",
  "entryHash": "c7e1..."
}
```

The genesis entry (first entry after signer boot, or after a log rotation) has `prevHash = "0000...0000"`. Each subsequent entry's `prevHash` equals the previous entry's `entryHash`. The `entryHash` is computed over the canonical JSON of the entry with `entryHash` itself omitted. The verify script re-computes each `entryHash` and checks the chain link-by-link; any break reports the entry index, expected vs. actual hash, and exits non-zero (CI-gatable).

What is NOT in Phase 1 (still Phase 2): shipping `signer-audit.jsonl` to a remote, append-only, tamper-proof sink (syslog, Loki, S3 with versioning + object lock). Local hash-chain tamper-evidence is cheap and Phase 1; remote shipping is operationally heavier and remains Phase 2. The hash chain is what makes the local file forensically useful even before remote shipping arrives — without it, an attacker with root could edit the local file and the operator would have no way to detect the tampering.

Tamper-evidence vs tamper-proofness: with the Phase 1 hash chain, this is *tamper-evident* (any selective edit breaks the chain), not *tamper-proof* (an attacker with root can still rewrite the entire file from scratch, including recomputing the chain). The defense against full-rewrite is operational: ship `signer-audit.jsonl` to a remote log collector in real time (Phase 2). Once an entry leaves the machine, it cannot be retroactively modified. The local hash chain catches the more common case of selective editing (changing one entry's value, destination, or status) without rewriting the whole file — which is what a sloppy or rushed attacker would attempt first.

v19.3-design-draft-3 operator follow-up (anti-truncation checkpoint): the hash chain detects mid-chain edits but NOT tail truncation — an attacker with write access can delete everything after entry N, and the chain still validates cleanly up to N (the verify script stops at the last entry and reports success). This is a real gap: truncation hides the most incriminating entries (the most recent sign RPCs, which are the ones an attacker would want to disappear).

The fix is a **monotonic counter + periodic checkpoint**:

1. **Monotonic counter:** each audit log entry includes a `seq` field that starts at 0 and increments by 1 for every entry. The verify script checks that `seq` is contiguous (0, 1, 2, ..., N) — any gap means entries were deleted from the middle, any repeat means an entry was duplicated. This is cheap (one integer per entry) and catches mid-chain deletion that the hash chain alone might miss if the attacker recomputes the chain after deletion.
2. **Periodic checkpoint:** every `SIGNER_AUDIT_CHECKPOINT_ENTRIES` entries (default 1000), the signer writes a checkpoint record to a SEPARATE file: `logs/signer-audit-checkpoint.jsonl`. Each checkpoint entry contains:

```
json {"ts":"2026-07-14T11:00:00.000Z","seq":1000,"entryHash":"9f3a...","entryCount":1000}
```

The checkpoint file is append-only and written by the same signer process that writes the main audit log. An attacker who wants to truncate the main log without detection must ALSO truncate the checkpoint file — and the verify script cross-checks that the last checkpoint's `seq` + `entryHash` match the corresponding entry in the main log. If the main log is shorter than the last checkpoint says it should be, truncation is detected.
3. **Verify script update:** `scripts/verify-signer-audit.ts` now:
 - Recomputes the hash chain (existing behavior).
 - Checks `seq` contiguity (new — catches mid-chain gaps).
 - Reads the checkpoint file, finds the last checkpoint, and verifies the main log contains at least that many entries AND the entry at `seq === checkpoint.seq` has `entryHash === checkpoint.entryHash` (new — catches tail truncation).
 - Reports a SEPARATE diagnostic for each failure mode: hash chain break, seq gap, or truncation.

Why this is still Phase 1 (not Phase 2): the checkpoint file is local, written by the same process, and verified by the same script. It's a few dozen lines of code on the write side and a few dozen on the verify side. The only thing that remains Phase 2 is shipping BOTH files (main log + checkpoint) to a remote sink — which is the strong defense against an attacker who can rewrite both local files. But the checkpoint makes local truncation detectable, which closes the gap the operator identified for a fraction of the cost of remote shipping.

7.3.5 Configuration model (tamper-resistance)

All policy values are loaded from env vars at signer boot. The signer has **no RPC method** to update them at runtime. To change a policy value, the operator must:

1. Edit the env var (or the allowlist JSON file).
2. Restart the signer process.
3. Re-unlock the vault (per § 9.2, restart always requires manual re-unlock).

This is intentional friction. The whole point of the signer-side policy is that a compromised web process cannot weaken it. If the web process could send a `set_policy` RPC to raise the cap or add an address to the allowlist, the policy would be worthless.

Env var	Default	Purpose
<code>SIGNER_MAX_TX_USD</code>	200 (mirrors <code>hotWalletCapUsd</code>)	Per-transaction cap. Sign RPCs above this value are rejected.
<code>SIGNER_MAX_WINDOW_USD</code>	500	Per-window rolling cap. Sum of signed values over <code>SIGNER_WINDOW_SEC</code> cannot exceed this.
<code>SIGNER_WINDOW_SEC</code>	300 (5 min)	Rolling window duration.
<code>SIGNER_DEST_ALLOWLIST</code>	<code>/etc/signer/dest-allowlist.json</code>	Path to the destination allowlist JSON file.
<code>SIGNER_AUDIT_LOG</code>	<code>logs/signer-audit.jsonl</code>	Path to the signer's independent audit log.
<code>SIGNER_PRICE_FEED_STALE_SEC</code>	60	Max age of a cached price before the signer rejects the sign RPC as <code>price unavailable</code> .

7.3.6 What this still does not prevent (honest limitation)

A compromised web process can still send a sign RPC for a **policy-compliant** transaction the operator did not authorize. Example: a \$150 swap to the Uniswap router, which passes the per-tx cap, the per-window cap, and the allowlist. The signer signs it. The operator's only defense is the independent audit log — they will see the entry in `signer-audit.jsonl` after the fact — and the Phase 1 hash chain (§ 7.3.4) ensures that entry cannot be retroactively edited to hide it.

Full mitigation requires Phase 3 intent-based signing: instead of taking raw `txData`, the signer takes an intent ("close position X", "rebalance 50/50") and constructs the tx itself. The web process cannot forge the destination or amount because it never sees them — it only sees the intent. This is a significant scope increase (the signer must understand trading semantics) and is correctly Phase 3.

The operator should treat § 7.3 as **raising the bar** from "any signed tx" to "a policy-compliant signed tx", not as fully closing the gap. The remaining gap is bounded by the per-window cap (max bleed rate) and

detected by the independent audit log (post-hoc forensics). This is the best achievable in Phase 1 without the full intent-based signing model.

7.3.7 Failed-unlock rate limiter scoping (v19.3 operator review point #3)

Question from operator: "o rate-limiter de tentativas falhas e o `SIGNER_MAX_WINDOW_USD` (§ 7.3.2) são escopados globalmente ao vault/signer, ou por origem? Se forem globais, um atacante batendo de um IP consegue travar o operador legítimo batendo de outro IP — um DoS auto-infligido pela própria proteção."

Answer (split by mechanism):

Mechanism	Scope	Rationale
Failed-unlock rate limiter (5 failures in 60s → 5min cooldown)	Per source IP (v19.3 fix)	Unlock attempts arrive via the HTTP API (<code>POST /api/vault</code>), which CAN receive concurrent requests from multiple IPs. Global scoping creates a self-inflicted DoS: an attacker from one IP can fill the failure counter and block the legitimate operator from another IP.
USD rolling window cap (<code>SIGNER_MAX_WINDOW_USD</code> , § 7.3.2)	Global per signer (unchanged)	The signer is single-writer (§ 9.4) — only the web process can call <code>sign</code> . There is no per-IP rotation attack at the signer layer. Per-IP scoping would weaken the cap without adding defense.
Per-transaction cap (<code>SIGNER_MAX_TX_USD</code> , § 7.3.1)	Global per signer (unchanged)	Same rationale — single writer, no identity rotation possible.

The bug was real and was manifesting in our own stress test data. The operator's hypothesis is confirmed by the 150s heterogeneous stress test results (§ 12.2):

- Per-profile results showed: `attacker` 27 fail / 26 err; `operator` 43 ok / 33 err; `lifecycle` 33 ok / 26 err; `interleaver` 19 ok / 7 fail / 27 err.
- Server log showed **1 rate-limit cooldown activation** (the `attacker` profile's 5 wrong-passphrase failures in 60s triggered the 5min cooldown).
- The test ran for 150s (2.5 min). Once the cooldown kicked in (likely within the first minute), it blocked ALL subsequent `unlock` attempts for the remaining ~2 min — regardless of which profile made them.
- The `operator` profile's 33 errors and the `lifecycle` profile's 26 errors are almost entirely "Unlock BLOCKED by rate limit" responses, not crypto failures of their own. Their successful `ok` counts (43 and 33) are the attempts that completed BEFORE the `attacker`-triggered cooldown started.

In other words: the `attacker` profile, by design trying wrong passphrases, DID deny service to the `operator` and `lifecycle` profiles in our own test. The bug the operator suspected was already in the data we collected — we just hadn't framed it that way.

Fix (v19.3.1 HOTFIX — SHIPPED, not Phase 1): per the operator's directive — "a correção do bug de auto-DoS deve sair do escopo da Fase 1 do signer e virar hotfix imediato e independente" — the rate limiter fix was implemented and tested as a standalone hotfix, NOT bundled into the 7-8 day Phase 1 signer isolation work. The hotfix ships in `src/lib/trading/wallet-crypto.ts` + `src/lib/trading/proxy-trust.ts` + `src/app/api/vault/route.ts` and is live in the current codebase.

Hotfix scope (3 sub-questions, all resolved):

Sub-question 1 — "De onde vem o `sourceIp`, e ele é confiável?"

Resolved (v19.3.2 fix to the v19.3.1 hotfix). The old `extractSourceIp()` in `/api/vault/route.ts` blindly trusted `x-forwarded-for` — a client-writable header. An attacker rotating a fake XFF value per request would bypass the per-IP rate limiter entirely, which is WORSE than the original global DoS bug (the old bug degraded availability; a bypassable rate limiter silently removes brute-force protection on the vault passphrase).

The fix lives in `src/lib/trading/proxy-trust.ts` and implements a two-mode trust model with a SPLIT pure decision function for testability:

- `resolveTrustedClientIp({ headers, peerAddress, proxySecret })` — pure function, no I/O, no Next.js runtime dependency. Exported so unit tests can call it directly with synthetic inputs (see scenarios 18 + 19 in `scripts/test-vault.ts`).
- `extractTrustedClientIp(req)` — async production wrapper. Reads the TCP socket peer address from an AsyncLocalStorage populated by the `http.Server.prototype.emit` monkey-patch in `src/lib/request-peer-capture.ts` (installed at boot via `src/instrumentation.ts`), then delegates to the pure function.

Next.js 16 App Router limitation (important context): Next.js 16 does NOT expose the TCP socket peer address to route handlers. The `connection()` function from `next/headers` returns `Promise<void>` — it's just a dynamic-rendering marker, NOT a connection-info accessor. (The `peer.address` API that existed briefly in Next.js 15.3 was REMOVED in 16.) `NextRequest.ip` was also removed. There is no supported way to read the non-spoofable TCP source IP from inside a route handler.

To get the peer address anyway, `src/instrumentation.ts` installs a monkey-patch on `http.Server.prototype.emit` (in `src/lib/request-peer-capture.ts`) that intercepts the 'request' event BEFORE Next.js wraps the Node `IncomingMessage` into a Web `Request`, reads `req.socket.remoteAddress`, and stores it in an `AsyncLocalStorage` (`src/lib/request-peer-als.ts`). Route handlers read it via `getRequestPeerAddress()`. This is the Next.js 16 App Router equivalent of Pages Router's `req.socket.remoteAddress` — non-spoofable for direct connections (the TCP handshake requires the real source IP).

Decision logic:

1. If `SIGNER_PROXY_SHARED_SECRET` env var is set AND the request carries `x-internal-proxy-secret` header matching the secret: we are behind a trusted reverse proxy. Trust XFF's leftmost entry as the real client IP. The proxy is responsible for overwriting any client-supplied XFF (Caddy: `header_up X-Forwarded-For {remote}`).
2. Otherwise (no secret configured, OR secret didn't match, OR the request didn't come through the proxy): use the TCP socket peer address (from the `AsyncLocalStorage`) directly as the per-IP key. The peer address is non-spoofable for direct connections — completing the TCP handshake requires the real source IP. This is exactly the non-spoofable identifier the per-IP rate limiter needs when there is no proxy relaying XFF.

v19.3.2 correction to v19.3.1 (the bug the operator caught): the v19.3.1 hotfix returned a fixed sentinel string `"direct-untrusted"` for ALL direct connections when no proxy secret was configured. That sentinel was a single shared bucket — exactly the same self-DoS shape as the original global `Date[]` bug, just renamed. For the deployment this project actually runs (bind `127.0.0.1`, no reverse proxy since v18), every real connection to the vault fell into the shared `"direct-untrusted"` bucket — the hotfix did not fix the bug for the actual topology; it only prepared the ground for a future behind-proxy deployment that does not exist yet. The v19.3.2 fix replaces the sentinel with the actual TCP socket peer address (captured via the `http.Server.prototype.emit` monkey-patch + `AsyncLocalStorage` — see the "Next.js 16 App Router limitation" paragraph above). This gives real per-IP isolation for the deployment running today, not just for a hypothetical future deployment.

The `"direct-untrusted"` sentinel is GONE. The only last-resort fallback (when `peer.address` is null — theoretically possible in some edge runtime adapters, never observed in Node.js runtime) is the

string `"unidentifiable-socket"`, named so it is visible in the audit log if it ever appears.

Why a shared secret for the proxy case (not a proxy-IP allowlist)? Even with `connection()` we only see the immediate TCP peer — which, behind a proxy, is the proxy itself. We cannot distinguish "request from Caddy on 127.0.0.1" from "request from an attacker on 127.0.0.1 who found the loopback bind" by IP alone. The shared secret is the application-layer equivalent of the proxy-IP allowlist: only a proxy that knows the secret can mark a request as trusted. Caddy config snippet:

```
reverse_proxy localhost:3000 {
  header_up X-Internal-Proxy-Secret {env.SIGNER_PROXY_SHARED_SECRET}
  header_up X-Forwarded-For {remote}
}
```

Known unresolved case (documented in runbook § 13.7): if the process runs behind NAT or Docker userland-proxy without terminating TLS/HTTP, `peer.address` collapses to a single gateway IP for multiple real origins. The per-IP rate limiter degrades to a shared bucket in that case — but the global aggregate cap (50 failures/1h across ALL IPs, a single array NOT subject to per-IP Map eviction) still catches distributed attacks, because it is independent of the per-IP key. This is a deployment-environment limitation, not a code bug. See § 13.7.

Sub-question 2 — "Crescimento não limitado do Map"

Resolved. The per-IP `Map<string, number[]>` has two bounds:

1. **TTL eviction:** on every access, `pruneFailedAttempts(ip)` drops timestamps older than `FAIL_WINDOW_MS` (60s). If an IP's array becomes empty AND its cooldown has expired, the IP is fully evicted from all three Maps (`failedAttemptsByIp`, `cooldownByIp`, `lastAccessByIp`).
2. **LRU cap:** `MAX_TRACKED_IPS = 10_000`. If the Map exceeds this, `evictLruIfNeeded()` evicts entries with the oldest `lastAccess` timestamp (that don't have an active cooldown — active-cooldown entries are NEVER evicted). At ~100 bytes per entry, 10k entries $\approx$ 1MB — a reasonable ceiling.

LRU eviction trade-off (acknowledged, operator review v19.3.2):

the eviction policy is "least recently accessed". An attacker with ENOUGH REAL SOURCE IPs (or a botnet) could, in theory, generate enough unique IPs to push their OWN earlier failure record out via LRU eviction — resetting their counter. This is significantly more expensive to exploit than the v19.3.1 sentinel model (which was

bypassable by header forgery, no real IPs needed): the attacker must either control many distinct source IPs (botnet, residential proxy network) or accept the cost of reconnecting from different source ports on the same host (which does NOT fool `peer.address`, since the kernel reports the same source IP across reconnects — only NAT/Docker userland-proxy can collapse distinct sources into one peer, see § 13.7). Two mitigations are in place:

- Active-cooldown entries are NEVER evicted (an attacker cannot reset their own cooldown by flooding the Map).
- The global aggregate cap (50 failures/1h across ALL IPs, a single array NOT subject to LRU eviction) still catches the aggregate pattern regardless of how the per-IP Map evicts.

The trade-off is accepted as defense-in-depth: per-IP catches single or low-cardinality attackers cheaply; the global cap catches distributed attacks that would also evade per-IP via eviction.

Sub-question 3 — "Camada dupla: per-IP não substitui um teto global"

Resolved. The hotfix implements BOTH layers:

- **Per-IP rate limiter:** 5 failures in 60s → 5min cooldown for THAT IP only. Resolves the self-DoS (attacker from one IP no longer blocks operator from another).
- **Global aggregate cap:** 50 failures in 1h across ALL IPs → 1min cooldown for ALL IPs (including unseen ones). Catches distributed attacks that per-IP alone would miss. The global cooldown is Milder than per-IP (1min vs 5min) — it's an alert-and-slow-down, not a full lockout, because a distributed attack shouldn't fully deny the legitimate operator.

Both cooldowns are checked on every unlock attempt; whichever is MORE restrictive (earlier expiry) wins. The audit log + notification context (`cooldownType`, `globalFailures`, `trackedIpCount`) distinguish which layer fired so the operator can tell single-source vs distributed.

Distinct notification event for the global cooldown (v19.3.2, operator review): the global aggregate cooldown firing is a SEVERITY-DIFFERENT signal from the per-IP cooldown. Per-IP means "one user / one attacker is hammering from one source"; GLOBAL means "many sources are failing in aggregate — likely a DISTRIBUTED attack or a botnet". If both fired the same generic `vault_rate_limited` event, the more severe distributed-attack signal would be lost in the noise of individual brute-force alerts.

The fix: `wallet-crypto.ts` emits a DISTINCT event type `vault_rate_limited_global` (added to `NotificationEventType` in `notifier.ts`) the moment the global cooldown ACTIVATES (crosses the

50-failure threshold), not only when a subsequent attempt is blocked. Operators can subscribe a louder / separate notification channel (phone-call tier, paged Slack channel) to `vault_rate_limited_global` independently from the generic `vault_rate_limited`. The per-event throttle is also shorter (60s vs 5min for per-IP) — distributed-attack bursts should surface every distinct wave, not be suppressed by a long dedupe window. The event message itself recommends the operator's response playbook (network-layer blocking or temporarily disabling the unlock endpoint), since the per-IP cooldown cannot resolve a distributed pattern.

Regression tests (5 scenarios in `scripts/test-vault.ts`, all passing — was 3 before v19.3.2):

- Scenario 15: "per-IP rate limit — attacker IP A does NOT block operator IP B". The core regression test. 5 failures from `198.51.100.10` → attacker IP in cooldown. `203.0.113.20` (operator) attempts unlock → NOT blocked, fails normally with wrong-passphrase error. Attacker IP STILL in cooldown after operator activity.
- Scenario 16: "global aggregate cap — distributed attack triggers global cooldown". 10 IPs × 5 failures each = 50 total → global cooldown activates. A NEW IP (never seen before) is blocked by the GLOBAL cooldown even though it has 0 per-IP failures.
- Scenario 17: "success clears per-IP failures but NOT global failures". Operator succeeds → operator IP failures cleared. Global failures NOT cleared (attacker still active). Attacker IP failures NOT cleared (only the succeeding IP clears).
- Scenario 18 (v19.3.2): "direct-IP isolation — two distinct direct IPs (no proxy) do NOT share a bucket". Calls the pure `resolveTrustedClientIp` with two distinct `peerAddress` values, asserts neither resolves to the `"direct-untrusted"` sentinel (which would be the v19.3.1 bug), asserts they resolve to DISTINCT strings matching their actual peer addresses. Then end-to-end: 5 wrong passphrases from `127.0.0.1` → cooldown on `127.0.0.1`. `192.168.1.42` (operator) attempts unlock → NOT blocked. This is THE regression test the operator required: "dois 'IPs diretos' distintos (sockets diferentes, sem proxy) não compartilham bucket".
- Scenario 19 (v19.3.2): "XFF forgery rejected — without proxy secret, forged XFF is ignored and peer address is used". Calls the pure `resolveTrustedClientIp` with three requests from the SAME peer address, each with a DIFFERENT forged `x-forwarded-for` value. Asserts all three resolve to the real peer address (not the forged XFF). Sanity: with the proxy secret configured AND present in the request, XFF IS trusted (proves the trusted-proxy path still works).

And: with the secret configured but missing from the request, the fallback is the peer address (proves the secret actually gates the XFF trust).

Test results: 20/20 passed, 0 failed (was 18/18 before v19.3.2; the 2 new scenarios all pass on first run). The suite is now self-contained — it truncates `WalletConnection` + `ExchangeConnection` tables at suite start (`hardCleanupBeforeSuite()`), so it works on a dirty DB without manual intervention. The operator's complaint — "test fails → clean DB manually → rerun → passes" pattern that happened 3 times in this thread — is resolved at the suite level, not the operator level.

Edge cases:

- **No proxy configured (the actual deployment topology of this project — bind 127.0.0.1 since v18):** `extractTrustedClientIp` uses `connection().peer.address` directly. For loopback connections this is `127.0.0.1` (or `::1`); for LAN connections it's the operator's LAN IP. Two distinct source IPs get distinct rate-limit buckets — the original self-DoS bug is fixed for the deployment running today, not just for a hypothetical behind-proxy future. (v19.3.1 returned a shared `"direct-untrusted"` sentinel here — that was the bug the operator caught; v19.3.2 replaces it with the real peer address.)
- **Behind a misconfigured reverse proxy** that strips `x-forwarded-for`: `extractTrustedClientIp` falls back to the proxy's own peer address (the immediate TCP peer) — all proxy traffic collapses to one bucket. Degraded but not bypassable. The operator should ensure the proxy sets XFF correctly (Caddy: `header_up X-Forwarded-For {remote}`).
- **`peer.address` is null** (theoretically possible in some edge runtime adapters, never observed in Node.js runtime): falls back to the string `"unidentifiable-socket"`. Named so it is visible in the audit log if it ever appears — the operator can investigate why `connection()` returned null.
- **NAT / Docker userland-proxy without TLS/HTTP termination** (documented in runbook § 13.7): `peer.address` collapses to a single gateway IP for multiple real origins. The per-IP rate limiter degrades to a shared bucket in this case, but the global aggregate cap (50 failures/1h, a single array NOT subject to per-IP Map eviction) still catches distributed attacks. This is a deployment-environment limitation, not a code bug.
- The slow-drip summary notifier (v19.2 in `wallet-crypto.ts`) still aggregates across all IPs — that's intentional, because the summary is a *meta-alert* ("N total attempts since last summary"), not a per-IP enforcement point.

Why the per-IP rate limit is still relevant in Phase 1 (signer-side):

**** in Phase 1, `wallet-crypto.ts` moves into the signer process. The rate limiter is enforced at the signer's `unlock` RPC handler. The web process still passes `sourceIp` in the `unlock` RPC for audit-log purposes, but the signer's rate-limit enforcement uses the socket peer address**** of the IPC connection — which, in the single-writer model (§ 9.4), is always the web process's own UID. This means in Phase 1, the per-IP rate limit primarily protects against multiple web processes (or a load balancer fanning out to the same signer) — not against the web process itself. The web process's own HTTP-layer rate limit (per source IP, via `extractTrustedClientIp` in `/api/vault/route.ts`) is the primary defense against external attackers; the signer-side rate limit is defense-in-depth. The hotfix ships the web-layer defense NOW (production is protected today); the signer-side defense-in-depth arrives with Phase 1.

8. Migration Path

Phase 1: Extract the signer (this design)

Goal: move `walletVault` into a separate process with a real trust boundary — not just process isolation, but a signer-side authorization policy that a compromised web process cannot bypass.

Steps:

1. Create `src/signer/main.ts` — the signer process entry point.
 - Imports `walletVault` from `wallet-crypto.ts` (the existing class, moved verbatim).
 - Creates the Unix socket, listens for connections.
 - Implements the JSON-RPC 2.0 dispatcher with the method allowlist.
 - Registers crash handlers (reuses `crash-logger.ts` with a different log prefix).
 - Registers the parent-disconnect handler (zero keys + exit on socket close).
 - **v19.3 (BLOCKING):** loads the authorization policy from env vars at boot (`SIGNER_MAX_TX_USD`, `SIGNER_MAX_WINDOW_USD`, `SIGNER_WINDOW_SEC`, `SIGNER_DEST_ALLOWLIST`, `SIGNER_AUDIT_LOG`, `SIGNER_PRICE_FEED_BINANCE_URL`, `SIGNER_PRICE_FEED_DEXSCREENER_URL`, `SIGNER_PRICE_FEED_STALE_SEC`, `SIGNER_WRITER_LEASE_TTL_SEC`, `SIGNER_WRITER_LEASE_RENEW_SEC`).The `sign` and `sign_typed_data` handlers enforce the policy BEFORE invoking `walletVault.sign()` — a rejected RPC never reaches the key. See § 7.3 for the full policy specification.
 - **v19.3 (BLOCKING):** opens the independent audit log (`logs/signer-audit.jsonl`) at boot and appends one JSON-Lines entry per sign RPC (accepted or rejected). The web process has no filesystem path to this file. See § 7.3.4.

- Imports `walletVault` from `wallet-crypto.ts` (the existing class, moved verbatim).
 - Creates the Unix socket, listens for connections.
 - Implements the JSON-RPC 2.0 dispatcher with the method allowlist.
 - Registers crash handlers (reuses `crash-logger.ts` with a different log prefix).
 - Registers the parent-disconnect handler (zero keys + exit on socket close).
 - **v19.3 (BLOCKING):** loads the authorization policy from env vars at boot (`SIGNER_MAX_TX_USD`, `SIGNER_MAX_WINDOW_USD`, `SIGNER_WINDOW_SEC`, `SIGNER_DEST_ALLOWLIST`, `SIGNER_AUDIT_LOG`, `SIGNER_PRICE_FEED_BINANCE_URL`, `SIGNER_PRICE_FEED_DEXSCREENER_URL`, `SIGNER_PRICE_FEED_STALE_SEC`, `SIGNER_WRITER_LEASE_TTL_SEC`, `SIGNER_WRITER_LEASE_RENEW_SEC`).
- The `sign` and `sign_typed_data` handlers enforce the policy BEFORE invoking `walletVault.sign()` — a rejected RPC never reaches the key. See § 7.3 for the full policy specification.
- **v19.3 (BLOCKING):** opens the independent audit log (`logs/signer-audit.jsonl`) at boot and appends one JSON-Lines entry per sign RPC (accepted or rejected). The web process has no filesystem path to this file. See § 7.3.4.
 - **v19.3-design-draft-3 (BLOCKING, operator review point #2):** each audit log entry includes `prevHash` + `entryHash` forming a hash chain from the genesis entry. `scripts/verify-signer-audit.ts` re-computes the chain and exits non-zero on any break (CI-gatable). This is local tamper-evidence; remote shipping (syslog/Loki/S3) remains Phase 2. See § 7.3.4.
 - **v19.3 (BLOCKING):** the signer fetches its own price feed (Binance + DexScreener, same sources as the web process but via the signer's own HTTP client) to compute USD values for the per-tx and per-window caps. The URL/source is fixed at boot from env vars — NOT a parameter of the `sign` RPC. See § 7.3.1. If the price fetch fails, sign RPCs are rejected (fail-closed).
 - **v19.3-design-draft-3 (BLOCKING, operator review point #1):** implements the writer LEASE with TTL — `acquire_writer`, `renew_writer` (every `SIGNER_WRITER_LEASE_RENEW_SEC`), auto-revoke on TTL expiry with zeroize-on-expiry. See § 9.4 for the full lease semantics and the explicit fail-closed consequence.
 - **v19.3-design-draft-3 (BLOCKING, operator review point #3):** the failed-unlock rate limiter uses a `Map<sourceIp, Date[]>` instead of a single global `Date[]`. Per-IP cooldown. See § 7.3.7. A new `scripts/test-vault.ts` scenario verifies: attacker-IP failures do NOT block operator-IP unlocks.

2. Create `src/lib/trading/signer-client.ts` — the web process's RPC client.

- Exports a `signerClient` object with the same method signatures as the current `walletVault` singleton.
- Each method sends an RPC and returns a Promise.

- Maintains a cached `getVaultStatus()` snapshot (refreshed on each call + on notifications).
 - Handles reconnection + degraded mode.
 - **v19.3:** surfaces `-32006 Policy violation` errors distinctly so the trading engine can log them as risk events (not just generic sign failures). The web process's `hotWalletCapUsd` check in `portfolio.ts` STAYS — it is now defense-in-depth, not the primary boundary.
- Update `src/app/api/vault/route.ts`:
 - Replace `import { walletVault } from "@lib/trading/wallet-crypto"` with `import { signerClient } from "@lib/trading/signer-client"`.
 - The route handler logic is unchanged — `signerClient.unlockAsync(passphrase, ip)` has the same signature.
 - Update `src/instrumentation.ts`:
 - After `registerCrashHandlers()`, call `spawnSigner()`.
 - `spawnSigner()` spawns the child process, waits for "SIGNER_READY", opens the socket connection, runs `health_check`.
 - Update `src/lib/trading/wallet-manager.ts`:
 - Replace direct `walletVault` calls with `signerClient` calls.
 - Update `src/lib/trading/engine.ts`:
 - The trading engine's `signTransaction` call goes through `signerClient.sign()`.
 - v19.3 (BLOCKING):** Create the default destination allowlist file at `/etc/signer/dest-allowlist.json` (or a project-local path for dev). The initial allowlist contains the DEX routers the trading engine actually uses (Uniswap V3 Router, 0x ExchangeProxy). An empty allowlist is valid (and means all sign RPCs are rejected) — this is the safe default for a fresh deployment.
 - v19.3 (BLOCKING):** Add a `scripts/test-signer-policy.ts` test script that exercises the authorization policy directly:
 - Sign RPC above per-tx cap → rejected with `-32006`.
 - Sign RPC that pushes rolling window over cap → rejected.
 - Sign RPC to non-allowlisted address → rejected.
 - Sign RPC to allowlisted address under both caps → signed.
 - Sign RPC with undecodable tx data → rejected.
 - Sign RPC when price feed is down → rejected.
 - Sign RPC carrying a `priceFeedUrl` or `price` field → rejected with `-32602 Invalid params` (verifies § 7.3.1 fixed-source rule).
 - Verify each accepted/rejected RPC produces an entry in `logs/signer-audit.jsonl`.
 - Verify the web process cannot write to `signer-audit.jsonl` (file permissions + path not exposed via any RPC).
 - **v19.3-design-draft-3:** verify the hash chain — `scripts/verify-signer-audit.ts` on the produced log exits 0; tampering with any entry exits non-zero.
 - **v19.3-design-draft-3:** verify per-IP rate limiting — 5 failed unlocks

from IP-A does NOT block IP-B from unlocking (regression test for § 7.3.7).

- **v19.3-design-draft-3**: verify lease auto-expiry — stop sending `renew_writer`, observe the signer zeroizes keys within TTL and the audit log records `writer_lease_expired`. See § 9.4.

9. **v19.3-design-draft-3 (BLOCKING, operator review point #2)**: Add `scripts/verify-signer-audit.ts` — reads `signer-audit.jsonl`, recomputes each `entryHash`, checks `prevHash` linkage. Exits 0 on intact chain, non-zero with a diagnostic (entry index, expected vs actual hash) on any break. CI-gatable. See § 7.3.4.

10. **v19.3-design-draft-3 (BLOCKING, operator review point #1)**: Add `scripts/test-signer-lease.ts` — tests the writer lease: (a) lease auto-expires when the holder stops renewing, (b) zeroize-on-expiry actually wipes keys, (c) a second connection can acquire the lease after expiry, (d) the audit log records the expiry event. See § 9.4.

Testing: the existing `scripts/test-vault.ts` suite should pass unchanged against the signer-client (it tests the public contract, not the implementation), with one new scenario for per-IP rate limiting (step 8 above). A new `scripts/test-signer-ipc.ts` should test the IPC protocol directly (socket connect, RPC round-trip, disconnect behavior). The new `scripts/test-signer-policy.ts` (step 8) tests the § 7.3 policy end-to-end. `scripts/verify-signer-audit.ts` (step 9) and `scripts/test-signer-lease.ts` (step 10) cover the v19.3-design-draft-3 additions.

Phase 1 is NOT approved for implementation until the operator signs off on § 7.3, § 9, and § 13. The original Phase 1 (process isolation only) is insufficient per the operator's v19.3 review point #3. The v19.3-design-draft-3 additions (lease TTL, hash chain, per-IP rate limit, runbook) are required per the operator's conditional approval.

Phase 2: Hardening

- **Request signing (HMAC):** at boot, the web process generates a random secret and passes it to the signer via `env` (not via the socket). Each RPC is signed with `HMAC-SHA256(secret, JSON.stringify(request))`. The signer verifies the signature before processing. This prevents a partially-compromised web process (e.g., SSRF that can write to the socket but not read env vars) from forging RPCs.
- **Signer-side notifier:** the signer holds its own notifier client and sends vault event notifications directly, not through the web process. This prevents a compromised web process from suppressing alerts.

- **Heartbeat canary:** the signer writes `logs/signer-heartbeat` every 5s with a timestamp. The web process checks the file on each `get_status` call; if the heartbeat is stale (>15s), it treats the signer as unhealthy even if the socket is still open (catches the case where the signer is hung but not crashed).
- **Remote audit log shipping (the Phase 2 portion of § 7.3.4):** ship `signer-audit.jsonl` to a remote, append-only, tamper-proof sink (syslog, Loki, S3 with versioning + object lock) in real time. The local hash chain (Phase 1) makes the file forensically useful on its own; remote shipping defends against full-rewrite attacks by an attacker with root on the signer host. Once an entry leaves the machine, it cannot be retroactively modified.

Phase 3: Future considerations (out of scope for v19.3)

- **Intent-based signing (full mitigation for § 7.3.6 limitation):** instead of signing raw tx data, the signer takes a higher-level intent ("close position X", "rebalance 50/50") and constructs the tx itself. This prevents a compromised web process from forging the destination or amount — the web process never sees them, only the intent. Requires the signer to understand trading semantics (position state, rebalance math). This is the full closure of the gap that § 7.3 only partially closes.
Note: the original draft of this document listed this as "Signer-side transaction validation" and framed it as a generic hardening. The v19.3 review clarifies that this is specifically the mitigation for "compromised web process sending policy-compliant but unauthorized sign RPCs" (§ 7.3.6). § 7.3 (Phase 1) raises the bar; Phase 3 intent-based signing closes the remaining gap.
- **Multiple signers / HSM backdoor:** for very high-value deployments, the signer could delegate to an HSM or hardware wallet for the actual signing. The IPC protocol stays the same; only the signer's internal implementation changes.

9. Open Questions for the Operator

v19.3: the operator's review provided explicit default positions on all five questions. This section is rewritten to reflect those defaults as the design's chosen answers (not just "recommendations"). Each subsection quotes the operator's position and explains how the design implements it. The operator should review this section before signing off on Phase 1 implementation — per the operator's directive: "o ponto 3 — política de autorização própria dentro do signer — eu trataria como pré-requisito de aprovação antes de começar a implementação, não um 'nice to have' do § 9."

9.1 Passphrase entry: web UI vs. signer TTY

Operator's default position:

"o processo web nunca deveria ver o texto plano, nem transitoriamente. Idealmente o signer possui o ciclo de vida inteiro da entrada de senha — mesmo que isso custe fricção de UX."

Options considered:

- **Option A (web UI — original draft's recommendation):** operator enters the passphrase in the dashboard, it crosses the IPC boundary via `unlock` RPC. ❌ Rejected — the web process sees plaintext, even transiently.
- **Option B (signer TTY):** the signer process prompts for the passphrase on its own stderr/stdin at boot (or on `unlock` notification from the web process). The web process sends `unlock` with no passphrase; the signer reads it from its TTY. ✅ Passphrase never crosses IPC.
- **Option C (hybrid — envelope encryption):** passphrase is entered in the web UI but is encrypted with a signer public key (generated at boot) before crossing IPC. The signer decrypts with its private key. ❌ The web process still sees the plaintext at the moment of UI input (before encryption). A compromised web process can still intercept it there. Does not satisfy the operator's "never see plaintext, even transiently" requirement.

Chosen answer (v19.3): Option B (signer TTY) as the default, with Option A available as an explicit opt-in for dev/low-value deployments.

Implementation:

- The signer reads the passphrase from its stdin (or a TTY if attached).
- The web process's `POST /api/vault` with `action: "unlock"` does NOT send a passphrase. Instead, it sends an `unlock` RPC with no passphrase, which causes the signer to print "Enter passphrase:" to its stderr and read from stdin.
- The operator must have a terminal attached to the signer process (e.g., via `tmux attach` or `systemctl attach`). This is the "UX friction" the operator explicitly accepted.
- For dev/low-value deployments, an env var `SIGNER_ALLOW_WEB_PASSPHRASE=1` opts back into Option A. The signer logs a loud warning at boot when this is set.
- The dashboard UI shows "Vault locked — enter passphrase on server terminal" when the signer is in TTY mode. The unlock input field is hidden.

Why this is worth the friction: the original motivating threat was a supply-chain compromise of the web process. If the passphrase crosses the web process's memory, that compromise can intercept it — and once the attacker has the passphrase, the signer-side authorization policy (§ 7.3) becomes the only remaining barrier. With TTY-mode passphrase entry, the attacker who compromises the web process can send sign RPCs (bounded by § 7.3) but cannot unlock a locked vault — they would need a separate compromise of the signer process or the operator's terminal session.

9.2 Signer restart policy

Operator's default position:

"fail-closed, sempre. Signer reiniciar nunca deveria restaurar chaves automaticamente — exige re-unlock manual, sem exceção."

Options considered:

- **Option A (original draft's recommendation):** auto-restart (max 3 retries with backoff), then fail open in LOCKED state. Operator must re-unlock. ❌ Rejected — "auto-restart" is fine, but "fail open in LOCKED state" is ambiguous; the original draft didn't explicitly require manual re-unlock after EVERY restart.
- **Option B (fail-closed, manual re-unlock always):** the signer can auto-restart, but the vault state is ALWAYS locked after a restart — no automatic restoration of decrypted keys, no exceptions. The operator must re-enter the passphrase (per § 9.1, on the signer TTY). ✅ Matches the operator's "fail-closed, sempre" directive.

Chosen answer (v19.3): Option B. Auto-restart is allowed (max 3 retries with backoff in 10 min), but EVERY restart — whether from crash, operator-initiated, or auto-restart — leaves the vault in LOCKED state. There is no "restore keys from previous session" code path. The signer's boot sequence always starts with `wallets = new Map()` (empty) and `unlocked = false`.

Implementation:

- The signer's `main.ts` boot sequence explicitly does NOT persist any unlock state across process boundaries. Decrypted keys live only in process memory; when the process exits (for any reason), they are gone.
- The web process's `signerClient` reconnect logic (§ 4.3) treats a reconnected signer as LOCKED — it does NOT attempt to re-send a cached passphrase. The operator must manually unlock via the signer TTY.
- The auto-restart cap is 3 retries in 10 minutes. If exceeded, the web process stops retrying, marks the signer as permanently down, halts the trading engine, and fires a critical alert.

- The audit log records every restart with the reason (crash, SIGTERM, auto-restart) so the operator can distinguish a transient glitch from a real bug.

9.3 Socket location

Operator's default position:

"0600 está certo; confirmar também que o diretório pai não permite outro usuário substituir o socket por symlink (vetor comum e frequentemente esquecido em sockets Unix compartilhados)."

Chosen answer (v19.3): the socket is created with mode `0600` (owner read/write only) AND the parent directory is verified safe against symlink swap at boot.

Implementation:

- **Socket path:** default `${XDG_RUNTIME_DIR}/signer.sock`, fall back to `/tmp/signer-<uid>.sock`. Configurable via `SIGNER_SOCKET_PATH`.
- **Socket file:** mode `0600`, owner = the user running the signer process. Created with `fs.mkdtempSync` for the parent dir if needed, then `net.createServer().listen(path)` — Node sets the socket file mode via `process.umask(0o077)` before `listen()`.
- **Parent directory symlink-swap check (v19.3, NEW):** at boot, the signer verifies the socket path's parent directory:
 1. `fs.lstatSync(parentDir)` — must NOT be a symlink. If it is, abort boot with error "socket parent dir is a symlink — possible symlink swap attack".
 2. `fs.statSync(parentDir).mode & 0o077` — must be `0` (no group/other permissions). If non-zero, abort boot with "socket parent dir is world/group-writable — another user could replace the socket".
 3. The signer records the parent directory's inode at boot. On any `fs.watch` event for the parent dir (rename, delete, attribute change), the signer treats it as a possible attack: zeros all keys, closes the socket, and exits with a crash log.
- For `/tmp/signer-<uid>.sock`, the signer creates a subdirectory `/tmp/signer-<uid>/` with mode `0700` owned by the user, and places the socket inside it. This avoids the `/tmp` is world-writable problem — the parent dir is the user-owned `0700` subdirectory, not `/tmp` itself.
- For `${XDG_RUNTIME_DIR}/signer.sock`, the parent (`/run/user/<uid>/`) is typically already `0700` and owned by the user on modern systemd distros. The signer still verifies this at boot rather than assuming.

9.4 Multiple web processes

Operator's default position:

"se o app web escalar horizontalmente, o signer precisa ser single-writer explícito — dois processos web tentando unlock/lock ao mesmo tempo no mesmo signer é exatamente o tipo de concorrência que motivou o CHECK 1 original, agora num novo componente."

v19.3 operator review point #1 (sign-off prerequisite):

*"`acquire_writer`/`release_writer` resolve a concorrência do CHECK 1 original, mas só se houver uma **lease com TTL**, não um lock que vive até `release_writer` explícito. Se o processo web que detém o writer lock crashar (exatamente a classe de evento que essa investigação inteira girou em torno de entender), o signer trava permanentemente esperando um **release** que nunca vem — um deadlock que exige restart manual do signer para destravar, ou seja, exatamente o cenário fail-closed que § 9.2 já aceita como aceitável, mas isso precisa estar explícito como consequência do design de lock, não como acidente descoberto depois. Confirmar: lease com timeout (ex.: renovação obrigatória a cada N segundos, expira sozinha se o dono sumir) — não lock indefinido."*

Chosen answer (v19.3-design-draft-3): the writer lock is now a **writer LEASE with mandatory TTL renewal**. The holder does not own the lock indefinitely — it must renew the lease periodically, and the lease auto-expires if the holder dies (crash, OOM, network partition, anything that prevents renewal). This is the only design that survives the failure mode the operator identified: a web process that crashes while holding the lock would otherwise leave the signer permanently deadlocked waiting for a `release_writer` that never arrives.

Implementation:

- The signer accepts multiple concurrent socket connections (for read-only operations like `get_status`, `health_check`), but only ONE connection at a time may hold the **writer lease**.
- The writer lease is acquired via an `acquire_writer` RPC (not to be confused with vault lock/unlock — this is an IPC-level lease). The first connection to call `acquire_writer` gets the lease; subsequent callers get `-32007 Writer busy` until the lease is released or expires.
- **Lease TTL (v19.3, mandatory):** the lease has a TTL of `SIGNER_WRITER_LEASE_TTL_SEC` (default 30s). The holder **MUST** renew the lease by sending a `renew_writer` RPC every `SIGNER_WRITER_LEASE_RENEW_SEC` (default 10s — i.e., renew at TTL/3 intervals). If the signer does not receive a `renew_writer` RPC within the TTL window, the lease is **automatically revoked**: the signer zeros all decrypted keys (per § 9.2 fail-closed principle),

closes the holder's socket connection, logs a `writer_lease_expired` event to the signer audit log (§ 7.3.4), and marks the vault as LOCKED.

The next `acquire_writer` caller can then take the lease.

- All state-mutating RPCs (`unlock`, `lock`, `create_wallet`, `delete_wallet`, `sign`, `sign_typed_data`) require the writer lease. If a connection without the writer lease calls one of these, the signer returns `-32007 Writer busy`.

- Read-only RPCs (`get_status`, `get_public_info`, `health_check`) do not require the writer lease.

- This prevents exactly the race the operator identified: two web processes cannot `unlock`/`lock`/`sign` concurrently against the same signer, because only one can hold the writer lease at a time.

- **v19.3-design-draft-3 operator follow-up (clean disconnect releases immediately):** the TTL is a BACKSTOP for the case where the holder

crashes uncleanly (no FIN packet, no `release_writer` RPC). A CLEAN disconnect — the web process closes the socket gracefully (SIGTERM, normal shutdown, `release_writer` RPC) — releases the lease

immediately, without waiting for the TTL to expire. This matters

for recovery time after a controlled web process restart: the operator restarts the web process, it reconnects, and can re-acquire the lease in <1s instead of waiting 30s for the old lease to time out. The

implementation: the signer's socket `close`/`end` event handler for the writer connection calls the same `revokeLease(holderConn)` function that the TTL expiry calls — zeroize keys, log `writer_lease_released` (distinct from `writer_lease_expired` so the audit log distinguishes clean vs. unclean release), mark vault LOCKED. The 30s TTL only fires when the holder disappears WITHOUT a clean socket close (crash, OOM, network partition).

- **The lease revocation is the same fail-closed consequence § 9.2 already accepts for signer restart.** A web process that crashes (the exact

failure class this entire investigation was about) loses its lease within 30s (unclean) or immediately (clean). The signer zeroizes keys and goes to LOCKED state. The operator must re-unlock via the signer TTY (§ 9.1). This is **not a new operational burden** — it's the same manual re-unlock that any signer restart already requires per § 9.2. Making it explicit here means it's a designed consequence, not an accident discovered in production. The audit log entry

`writer_lease_expired` (unclean) or `writer_lease_released` (clean) lets the operator distinguish "web process crashed" from "signer crashed" — both lead to LOCKED, but the remediation is different (restart the web process vs. restart the signer).

- The web process's `signerClient` acquires the writer lease at boot (or on reconnect) and starts a `renew_writer` timer immediately. If a second web process starts and tries to acquire the writer lease, it

fails — the operator must either run one web process or use a load balancer that routes all vault operations to a designated "writer" process. If the writer web process dies, the lease expires within 30s and a designated failover web process can take over (after the operator re-unlocks — failover does NOT bypass § 9.2).

Lease renewal failure handling on the web side:

- If the web process's `renew_writer` RPC fails (signer unreachable, socket error, etc.), the web process treats it as a lost lease: stops sending `sign` RPCs, fires a `vault_locked` notification, and surfaces the failure in the dashboard. The web process does NOT attempt to re-acquire the lease automatically — that would mask the underlying failure (signer crash, network partition, etc.). The operator must investigate and manually re-acquire after confirming the signer is healthy.
- If the web process's `renew_writer` RPC succeeds but the response indicates the lease is no longer held (e.g., the signer revoked it because a higher-priority process took over — not currently supported in Phase 1, but reserved for future multi-writer-arbitration scenarios), the web process immediately enters degraded mode and halts the trading engine.

Out of scope for v19.3 implementation: the current deployment is a single web process, so the writer lease is held continuously and never contended. The lease infrastructure (TTL, renewal, auto-revocation, zeroize-on-expiry) is built into Phase 1 so that scaling later doesn't require a protocol change. A `scripts/test-signer-lease.ts` test verifies: (a) lease auto-expires when the holder stops renewing, (b) zeroize-on-expiry actually wipes keys (the signer refuses subsequent `sign` RPCs until re-unlock), (c) a second connection can acquire the lease after expiry, (d) the audit log records the expiry event.

9.5 Signer dependency surface

Operator's default position:

"mínima e com versão fixa, especificamente porque esse processo agora é a fronteira de confiança real do sistema — reforça o que já está no `SECURITY.md`."

Chosen answer (v19.3): the signer has a **minimal, pinned dependency tree**, auditable in isolation from the web process. This is now more important than in the original draft because the signer is the trust boundary (per § 7.3), not just a signing oracle.

Implementation:

- Signer dependencies (allowed list):

- `ethers` (transaction signing) — pinned to exact version in `package.json`, hash recorded in `SECURITY.md`.
- `@prisma/client` (DB access for wallet/exchange rows) — pinned.
- Node built-ins (`net`, `crypto`, `fs`, `child_process`, `os`).
- The signer's own `crash-logger.ts` (shared with the web process, but only the crash-logging functions — no imports from `trading/*`).

- Signer dependencies (forbidden list):

- Anything from `src/lib/trading/*` except `wallet-crypto.ts` (the vault class). Specifically: no `engine.ts`, no `portfolio.ts`, no `risk-manager.ts`, no `notifier.ts` (Phase 2 will move notifier access to the signer, but with a minimal client, not the full module).
- Anything from `src/app/*` (no Next.js coupling).
- No `axios`, `node-fetch`, or other HTTP clients — use Node's built-in `fetch` (available in Node 18+). This is for the signer's independent price feed (§ 7.3.1).
- No `lodash`, `underscore`, or other utility libraries. Use built-in language features.
- **Pinning enforcement:** the signer's `package.json` (separate from the web app's) uses exact versions (no `^` or `~`). A `scripts/audit-signer-deps.ts` script runs `npm audit --omit=dev` on the signer's dependency tree and fails CI if any vulnerability is found.
- **Audit:** the signer's full dependency tree (including transitive deps) is dumped to `SECURITY.md` on every release. The operator reviews the diff before signing off.

10. Testing Strategy

10.1 Unit tests (existing, unchanged)

`scripts/test-vault.ts` scenarios 1-14 test the vault's public contract (unlock, lock, rate limit, auto-lock, atomicity, TOCTOU, notification throttle, slow-drip summary, source IP). These tests run against `walletVault` directly, which in Phase 1 moves into the signer. The tests import `walletVault` from the signer module — they test the vault class in isolation, not the IPC layer.

10.2 IPC integration tests (new)

`scripts/test-signer-ipc.ts` — tests the IPC protocol + process lifecycle:

1. **Boot:** spawn signer, verify `health_check` passes within 10s.
2. **RPC round-trip:** unlock → get_status → sign → lock, verify each response.

3. **Method allowlist:** send `export_key` RPC, verify `-32601 Method not found`.
4. **Malformed JSON:** send `not-json\n`, verify `-32700 Parse error` + connection stays open.
5. **Concurrent requests:** send 10 `get_status` RPCs in parallel, verify all 10 responses match (by correlation ID).
6. **Parent disconnect:** kill the web-side socket, verify the signer exits within 2s + writes a crash-log entry.
7. **Signer crash:** trigger an `uncaughtException` in the signer, verify `crash-*.log` is written + web process detects disconnect + marks vault locked.
8. **Reconnect:** after simulated crash, verify the web process spawns a new signer + `health_check` passes.
9. **IPC timeout:** send an RPC with a 1ms timeout, verify `TimeoutError` + vault marked locked.

10.3 Stress test (existing + new)

The heterogeneous stress test (`scripts/stress-test-heterogeneous.ts`) should be re-run after Phase 1 implementation, now hitting the IPC layer instead of the in-process vault. This verifies the signer doesn't introduce new race conditions or resource leaks under concurrent load.

11. Implementation Estimate

Component	Effort	Notes
<code>src/signer/main.ts</code> (signer process)	2 days	Move vault class, implement JSON-RPC dispatcher, method allowlist, crash handlers, disconnect handler. Includes § 7.3 signer-side policy enforcement (per-tx cap, rolling window cap, allowlist, independent audit log with hash chain, own price feed).
<code>src/lib/trading/signer-client.ts</code> (web-side client)	1 day	RPC client with reconnection, cached status, timeout handling, writer lease acquisition + renewal timer.
<code>src/instrumentation.ts</code> (spawn signer at boot)	0.5 day	spawn + wait for ready + <code>health_check</code> .
Update API routes + wallet-manager + engine	0.5 day	Swap <code>walletVault</code> → <code>signerClient</code> .
Writer lease + TTL + zeroize-on-expiry	0.5 day	<code>acquire_writer</code> / <code>renew_writer</code> / auto-revoke logic in signer; <code>renew_writer</code> timer in client. See § 9.4.
Hash chain + <code>verify-signer-audit.ts</code>	0.5 day	<code>prevHash</code> / <code>entryHash</code> on each audit log entry; CI-gatable verify script. See § 7.3.4.
<code>scripts/test-signer-ipc.ts</code> (integration tests)	1 day	9 scenarios in § 10.2.

<code>scripts/test-signer-policy.ts</code> (policy tests)	1 day	9 scenarios in § 8 step 8, including the v19.3-design-draft-3 additions (price-feed-source-fixed, hash chain, per-IP rate limit, lease auto-expiry).
<code>scripts/test-signer-lease.ts</code> (lease tests)	0.5 day	4 scenarios in § 8 step 10.
§13 Operator Runbook validation	0.5 day	Walk through each runbook procedure against a staging deployment; confirm steps are accurate and complete.
Total Phase 1	7-8 days	Up from 4-5 in the original draft and 6-7 in draft-2. The extra day over draft-2 covers the writer lease + TTL, the hash chain + verify script, and the runbook. (The per-IP rate limit refactor is no longer in Phase 1 — it shipped independently as the v19.3.1/v19.3.2 hotfix, see § 7.3.7.)

Phase 2 (HMAC request signing, signer-side notifier, heartbeat canary, remote audit log shipping) is an additional 2-3 days and can be deferred until after the crash investigation is fully closed.

12. Relationship to the Crash Investigation

The operator's directive: "se a causa raiz acabar sendo algo estrutural (ex.: uma race real no acesso ao Map de wallets), isso é exatamente o tipo de bug que o isolamento em processo separado também deveria herdar/mitigar — e você quer saber disso *antes* de desenhar o protocolo de IPC, não depois."

This design addresses that concern in two ways:

- The signer inherits the single-threaded Map-access semantics of the current vault.** If the crash was caused by a race in `walletMap` access, the same race exists in the signer — but now it's isolated to the signer process, so a crash takes down only the signer (not the web server). The web process detects the disconnect, marks the vault locked, and restarts the signer. The operator sees "signer crashed" instead of "server died silently". This is strictly better diagnostics.
- The crash handler (§ 4.3, § 4.4) is symmetric.** Both processes use the same `crash-logger.ts` with synchronous file logging. If the signer crashes, we get a stack trace in `logs/signer-crash-*.log`. If the web process crashes, we get one in `logs/crash-*.log`. Either way, the operator has a real trace to root-cause, not a silent death.
- If the heterogeneous stress test (Check 1) reproduces the crash organically, implementation should be PAUSED until the root cause is fixed.** There's no point building an IPC protocol on top of a vault that

has a race condition — the signer would just inherit the crash. The stress test result (reported alongside this design) determines whether we proceed to Phase 1 implementation or fix the underlying bug first.

12.1 Stress test results (reported alongside this design)

Retrospective audit (bonus finding): the dev DB had **0 wallets and 0 exchanges** when the previous stress test ran. This means every "wrong passphrase" unlock returned **200 (empty-vault success)**, not 401 (crypto failure). The crypto failure path (PBKDF2 derivation + GCM auth fail + `recordFailedAttempt` + `notifyVaultEvent(vault_unlock_failed)`) was **never exercised**. The previous stress test's 35,570 requests with 0 crashes is still useful evidence (rules out resource exhaustion at that volume), but it did NOT test the path most likely to trigger the original crash. The in-process test suite (`test-vault.ts`) was NOT affected — all 14 scenarios use `makeWallet()` to create real wallets before testing.

CHECK 1 (heterogeneous stress test): NEGATIVE. The production server survived 48 seconds of heterogeneous concurrent load (8 workers, 4 profiles) with **no organic crash**. 94 requests completed before the debug trigger fired at the 40% mark. The 4 worker profiles exercised:

- **attacker:** wrong-unlock $\times$ 3 $\rightarrow$ lock (crypto failure path: PBKDF2 + GCM auth fail + rate-limit + notify throttle)
- **operator:** right-unlock $\rightarrow$ GET status $\rightarrow$ lock (full DB read + decrypt)
- **lifecycle:** create wallet $\rightarrow$ right-unlock $\rightarrow$ lock $\rightarrow$ delete (DB mutations during concurrent vault access)
- **interleaver:** wrong-unlock $\rightarrow$ create $\rightarrow$ right-unlock $\rightarrow$ lock $\rightarrow$ delete (the specific sequence the operator said caused the original crash)

The server log confirms the crypto failure path was exercised: `decryptSecret failed: Error: Unsupported state or unable to authenticate data` (GCM auth tag failure), `Unlock FALHOU – N blob(s) não descriptografados`, `notifyVaultEvent(vault_unlock_failed) THROTTLED`.

CHECK 2 (crash handler armed under load): PASSED. The debug crash endpoint (`/api/debug/crash-test`, gated by env var) was called mid-stress-test with 147 requests in flight. The crash handler fired and produced a `crash-*.log` with a full stack trace:

```
Error: DELIBERATE CRASH TEST (uncaughtException) – pid=30558 uptime=61.3s
  at Timeout._onTimeout (.next/server/chunks/[root-of-the-server]__*.js:1:4246)
  at listOnTimeout (node:internal/timers:605:17)
  at process.processTimers (node:internal/timers:541:7)

ACTIVE HANDLERS / TIMERS:
  unref'd handles: [Socket, Socket, Server, Socket, Socket, Socket, Socket,
    Socket, Socket, Socket, Socket, Socket, Socket]
  process.listenerCount(uncaughtException)=3
```

The 13 active Socket handles confirm the server was under concurrent load when the handler fired. `process.listenerCount(uncaughtException)=3` shows our handler is registered alongside Next.js's own (Node calls all listeners).

Secondary finding — event loop congestion under load: there was a **3.5-second delay** between the HTTP response and the actual crash. The debug endpoint's `setTimeout(100ms)` callback was delayed by event loop congestion from the 8 concurrent workers. This is expected Node.js behavior (timers are not real-time) but has implications for the signer isolation design: the IPC timeout (§ 4.5) must account for multi-second event loop latency under load. A 5s timeout is too tight; 15s is recommended for non-signing operations, 30s for `unlock` (which does $\text{PBKDF2} \times N$ wallets).

Combined conclusion: CHECK 1 negative + CHECK 2 passed = the silent-crash investigation can be **downgraded to MONITOR without reservation**, per the operator's criteria. The heterogeneous recipe did not reproduce the crash in production build. The crash handler is confirmed armed and firing under concurrent load. **Phase 1 implementation of the signer isolation can proceed.**

12.2 v19.3 follow-up: clean 150s run + port-collision hypothesis

Operator's v19.3 review point #1:

"48 segundos / 94 requisições antes do gatilho de debug disparar é uma amostra ordens de magnitude menor que os 120s / 35.570 requisições do teste homogêneo anterior. ... para eu tratar isso com o mesmo peso de evidência que o teste anterior, eu rodaria uma passada adicional — mesma receita heterogênea, sem o gatilho de debug desta vez — por uns 120-180s corridos, só para ter um resultado limpo de mesma ordem de grandeza."

CHECK 1 clean rerun (150s, no debug trigger, RUN_CHECK_2=0):

Metric	Prior 48s run (with CHECK 2)	This 150s run (CHECK 1 only)
Duration	48s (killed by CHECK 2 at 40%)	150.1s (full duration)
Total cycles	94	247 (2.6× larger)

Workers	8 (CONCURRENCY=2)	16 (CONCURRENCY=4)
Crypto path exercised	YES (5 baseline wallets)	YES (5 baseline wallets)
Organic crash	NO	NO
New crash-*.log during stress loop	0	0
CHECK 2 trigger	Fired (killed server)	Skipped (RUN_CHECK_2=0)

Per-profile stats (150s run):

- attacker: sent=59 ok=6 fail=27 err=26 (wrong-passphrase failures + rate-limit)
- operator: sent=76 ok=43 fail=0 err=33
- lifecycle: sent=59 ok=33 fail=0 err=26
- interleaver: sent=53 ok=19 fail=7 err=27
- **Total: 247 cycles, ~7-8 effective HTTP req/s** (dominated by PBKDF2 600k iterations per unlock)

Server log analysis (994 lines):

- 775 `decryptSecret failed: Error: Unsupported state or unable to authenticate data` (expected — attacker profile's wrong passphrases hitting PBKDF2/GCM auth)
- 46 `Unlock FALHOU – N blob(s) não descriptografados` (expected)
- 1 rate-limit cooldown activation (5 failures in 60s → 5min cooldown — working as designed)
- 43 `notifyVaultEvent(vault_unlock_failed) THROTTLED` (notification throttle working)
- 51 successful `Vault DESBLOQUEADO`, 35 `Vault BLOQUEADO`, 36 `Wallet created` (normal operation)
- **Zero unexpected warnings. Zero empty-vault warnings (5 baseline wallets pre-created).**

Important caveat on sample size: the operator's framing was "mesma ordem de grandeza" as the prior 120s/35,570-request homogeneous test. This run is NOT same order of magnitude in raw request count (247 cycles $\approx$ 1,000-1,200 HTTP calls vs 35,570). The throughput is ~2 orders of magnitude lower because each heterogeneous cycle is dominated by PBKDF2 key derivation and serializes on the vault's single in-memory secret state. The homogeneous test was hitting lightweight GET endpoints.

However, this run IS the correct domain for the crash recipe being tested (wrong-unlock → create → right-unlock → lock → delete — the original silent-crash sequence). The 150s run hit 247 cycles of that exact recipe, vs the prior 48s run's 94 cycles — a $2.6\times$ larger sample of the relevant operation sequence. The operator may still want an even longer run (300s+) for a truly same-order-of-magnitude sample; this 150s run is offered as the minimum that satisfies the "120-180s" target.

Script false-positive note: the stress test script initially returned exit code 2 (CHECK 1 POSITIVE) due to a logic bug in the `organicCrash` determination when `RUN_CHECK_2=0` — the `serverDied` flag was set by the 'exit' handler regardless of whether the exit happened mid-stress or post-stress (script's own SIGTERM). The subagent running the test caught

this, reported it honestly rather than re-running until it gave the "right" answer, and the script was fixed (added `serverDiedDuringStress` flag that only fires if the exit happens before `stressLoopEndedAt`). The corrected determination confirms: **CHECK 1 NEGATIVE** — the server survived the full 150s of heterogeneous load with zero organic crashes.

Operator's v19.3 review point #2 (port-collision hypothesis):

"será que pelo menos parte das quedas silenciosas originais eram colisão de processo/porta durante hot-reload do Turbopack (uma instância antiga ainda de pé, ou uma recompilação no meio de uma request), e não um bug de lógica no vault?"

Investigation findings (full report at [download/port-collision-investigation.md](#)):

Evidence searched	Result
Direct <code>EADDRINUSE</code> matches in project source/logs/worklog	0
<code>address already in use</code> matches	0 (2 unrelated hits in <code>signer-isolation-design.md</code> for <code>ethers/@prisma/client</code>)
Documented "two launches collided" event	1 — <code>worklog.md:1515</code> (script-level collision in <code>next start</code> , NOT <code>next dev</code> HMR)
Orphaned-child / EPIPE cleanup artifacts	3 — <code>worklog.md:1570/1572/1580</code> (all <code>next start</code> , not dev)
<code>listen()</code> frames in any <code>crash-*.log</code>	0 (all 4 crash logs are either deliberate debug triggers or EPIPE cleanup)
Current machine state (<code>lsof -i :3000/:3100/:3200</code>)	clean — no listeners

Verdict: INDETERMINATE. The hypothesis is structurally plausible and consistent with the operator's "doesn't reproduce in production" framework (`next start` has no HMR, no concurrent compilations), but no direct `EADDRINUSE` evidence exists in any captured log. The investigation cannot distinguish port-collision-during-HMR from three other dev-mode-specific candidates:

- 1. **v18b Turbopack circular-import** — the leading theory (documented in worklog)
- 2. **v15 sandbox resource limits**
- 3. **Other dev-mode bugs** (e.g., Turbopack memory leaks during fast recompiles)

All four share the same observable signature: silent crash in `next dev`, no reproduction in `next start`. The crash-logger added in v19.1 will produce a real stack trace the next time any silent crash recurs — if that

trace shows `EADDRINUSE` or a `listen()` frame, the hypothesis is CONFIRMED for that specific crash; otherwise it is REFUTED for that crash.

Operational hygiene fix: a new pre-flight helper `scripts/check-port-orphan.sh` was created. Operators should run it before every `next dev` / `next start` / stress-test invocation. It checks `lsof -i :3000/:3100/:3200` for listeners, prints the PID/command/start-time of any orphan, and asks for confirmation before killing. This doesn't prove or refute the hypothesis, but it closes the operational gap that made the collision possible in the prior session.

Impact on MONITOR status: the port-collision hypothesis does NOT change the silent-crash investigation status. It remains at **MONITOR** — the crash-logger is armed (CHECK 2 confirmed), the heterogeneous recipe did not reproduce the crash in 150s of production stress (CHECK 1 confirmed), and the next time any silent crash recurs (in dev OR production), the crash log will tell us whether it's `EADDRINUSE`, a Turbopack issue, or something else entirely. The `setImmediate` fix in `wallet-crypto.ts` remains PROVISIONAL (no captured stack trace points to any specific cause).

13. Operator Runbook (v19.3-design-draft-3, operator review point #5)

"Trocar de estratégia/adicionar um router novo (ex.: expandir de Ethereum/BSC para Solana) vai exigir restart + re-unlock manual com alguma frequência normal de operação, não só em emergência. Isso é a escolha certa de segurança, mas peço que vire uma seção de runbook operacional explícita (passo a passo: editar JSON → restart signer → digitar passphrase → confirmar allowlist ativa), não fique só implícito no texto de design — quem for operar isso daqui a 3 meses não vai lembrar o raciocínio, só vai precisar do procedimento."

This section is the operator-facing runbook. It assumes the signer is already deployed and running in production (i.e., Phase 1 implementation is complete). Every procedure here involves restart + re-unlock — that is the designed friction, not an oversight. The operator should bookmark this section.

13.1 Adding a new DEX router to the destination allowlist

When to use: you want the trading engine to be able to route swaps through a new DEX router (e.g., adding Camelot on Arbitrum, or a new Uniswap V4 deployment). Until the router is in the allowlist, sign RPCs to it are rejected with `-32006 destination_not_in_allowlist`.

Steps:

1. **Identify the router address and chain.** Confirm the address on the DEX's official docs (NOT a search-engine result — type the official domain manually). Record: `address`, `chainId`, `label`, and the `methods` you want to permit (use `["*"]` only if you accept all methods on this router; otherwise list the specific selectors, e.g., `["exactInputSingle", "exactInput"]`).

2. **Edit the allowlist JSON file** at the path pointed to by `SIGNER_DEST_ALLOWLIST` (default `/etc/signer/dest-allowlist.json`). Append the new entry to the `entries` array. Bump the `version` field. Update `loadedAt` to the current ISO timestamp (this will be overwritten by the signer at boot, but setting it now makes the diff self-documenting).

```
sh # ALWAYS work on a copy first, then atomically move into place. cp /etc/signer/dest-allowlist.json /etc/signer/dest-allowlist.json.bak.$(date +%Y%m%d-%H%M%S) $EDITOR /etc/signer/dest-allowlist.json # Validate JSON before installing: python3 -c "import json; json.load(open('/etc/signer/dest-allowlist.json'))" \ && echo "JSON valid" || echo "JSON INVALID - abort and restore from .bak"
```

3. **Verify the new entry is what you expect** before restarting:

```
sh jq '.entries[-1]' /etc/signer/dest-allowlist.json
```

4. **Restart the signer process.** How depends on the deployment:

- systemd: `sudo systemctl restart signer`
- pm2: `pm2 restart signer`
- manual: kill the PID, re-run the boot command

The signer refuses to start if the allowlist JSON is malformed (fail-closed per § 7.3.3). If it does, restore from the `.bak` file from step 2 and try again.

5. **Re-enter the passphrase on the signer TTY.** Per § 9.2, restart always requires manual re-unlock — there is no auto-restore. Attach to the signer's terminal:

```
sh # If running under systemd with a TTY: sudo systemctl attach signer # Or if running under tmux: tmux attach -t signer
```

The signer prints `Enter passphrase:`. Type it. The signer logs `Vault DESBLOQUEADO` and is now operational.

6. **Confirm the allowlist is loaded and active.** Run the signer's health check, which now includes an allowlist summary:

```
sh # From the web host (the only host that can connect to the socket): sudo -u <web-user> scripts/signer-cli.sh get_public_info
```

The response should include `allowlistVersion`, `allowlistEntryCount`, and `allowlistLoadedAt`. Verify `allowlistVersion` matches what you set in step 2, and `allowlistEntryCount` is one higher than before.

7. **Verify the new router accepts a sign RPC.** Use the signer policy test script in dry-run mode (does NOT actually broadcast the tx):

```
sh tsx scripts/test-signer-policy.ts --dry-run --to <new-router-address> --value 1
```

Expect: `status: signed (dry-run)`. If you get `rejected: destination_not_in_allowlist`, the address in step 2 doesn't match the address in step 7 (checksum mismatch is the usual cause).

8. **Notify any other operators** (Slack, PagerDuty, whatever your channel is) that the allowlist was updated, with the version number and the new entry label. This is operational hygiene, not a design requirement — but the audit log (§ 7.3.4) is what makes this after-the-fact verifiable, and telling humans in advance makes incident response faster if the new router turns out to be misconfigured.

13.2 Removing a router from the allowlist

When to use: a router is compromised, deprecated, or you no longer want the trading engine to use it. Until removed, sign RPCs to it continue to be policy-compliant.

Steps: same as § 13.1, but in step 2 you delete the entry from the JSON instead of adding one. The rest of the procedure (restart, re-unlock, verify, notify) is identical. After step 6, run the dry-run policy test against the removed router and confirm it now returns `rejected: destination_not_in_allowlist`.

13.3 Changing the per-tx or per-window USD cap

When to use: you want to raise (or lower) the maximum value of a single signed transaction, or the rolling-window total. Common reasons: portfolio grew and 200/500 is too tight; portfolio shrank and you want to tighten; a new strategy requires larger swaps.

Steps:

1. **Edit the signer's env file** (typically `/etc/signer/signer.env` or your process manager's env var config). Change:
 - `SIGNER_MAX_TX_USD=200` → new per-tx cap (in USD).
 - `SIGNER_MAX_WINDOW_USD=500` → new per-window cap (in USD).
 - `SIGNER_WINDOW_SEC=300` → new window duration (in seconds), if you also want to change the window length.
2. **Restart the signer** (same as § 13.1 step 4).
3. **Re-enter the passphrase** (same as § 13.1 step 5).

4. Confirm the new caps are loaded:

```
sh sudo -u <web-user> scripts/signer-cli.sh get_public_info
```

The response includes `maxTxUsd`, `maxWindowUsd`, `windowSec`. Verify they match what you set in step 1.

5. Verify the new caps are enforced by running the policy test:

```
sh tsx scripts/test-signer-policy.ts --dry-run --value <new-cap-plus-1>
```

Expect: `rejected: tx_cap_exceeded`. Then:

```
sh tsx scripts/test-signer-policy.ts --dry-run --value <new-cap-minus-1>
```

Expect: `status: signed (dry-run)` (assuming the rolling window has room — if not, you'll see `rejected: window_cap_exceeded`, which is also correct behavior).

6. Update the web process's `hotWalletCapUsd` in

`src/lib/trading/config.ts` to match (this is now defense-in-depth, not the primary boundary, but keeping them in sync avoids confusing log messages where the web side says "OK" and the signer side says "rejected"). Redeploy the web process. No signer restart is needed for the web-side change.

7. Notify other operators with the new cap values and the reason for the change.

13.4 Expanding to a new chain (e.g., adding Solana)

When to use: you want the trading engine to operate on a chain that isn't currently supported (the initial deployment is Ethereum mainnet + EVM L2s). This is the largest routine operation because it involves both the allowlist (§ 7.3.3) AND the signer's price feed (§ 7.3.1) — Solana tokens don't have prices on Binance/DexScreener-EVM.

Pre-flight (do this BEFORE touching the signer):

1. Confirm the signer's `ethers` dependency supports the new chain's

signing algorithm. For Solana, the answer is no — Solana uses Ed25519, not secp256k1. This means a Phase 1 signer (built on `ethers`) CANNOT sign Solana transactions. Expanding to Solana is a Phase 3-scale change (new signing library, new key format, new price feed source), not a § 13.4 runbook operation. Treat it as a separate engineering project.

2. For EVM-compatible chains (Base, Arbitrum, Optimism, BSC, etc.):

no signer code changes are needed — only the allowlist + a price feed source that covers the chain's tokens. Continue to the steps below.

Steps (EVM-compatible new chain only):

1. **Confirm the new chain's `chainId`** (e.g., Base = 8453, Arbitrum = 42161, BSC = 56). The allowlist entries use `chainId` to scope which entries apply to which chain.
2. **Confirm the price feed source for the new chain's tokens.** The signer's default price feed is Binance (for blue chips) + DexScreener (for long tail). DexScreener covers most EVM chains already. If the new chain's tokens aren't on DexScreener, you need a new price source — add it via env vars (`SIGNER_PRICE_FEED_<SOURCE>_URL`). The signer's price-fetch code must be updated to query the new source. This is a code change, not just a config change.
3. **Add the new chain's DEX routers to the allowlist** (§ 13.1), with the correct `chainId`.
4. **Update the web process's trading engine** to actually trade on the new chain (token-selector, dex-screener query, etc.). This is outside the signer's scope — the signer just signs whatever tx the web process sends, as long as it passes policy.
5. **Restart the signer** (§ 13.1 step 4) and **re-enter the passphrase** (§ 13.1 step 5).
6. **Verify with a dry-run sign RPC** to a router on the new chain (§ 13.1 step 7). The signer should accept it (assuming the router is in the allowlist, the value is under the cap, and the price feed returned a valid USD price for the input token).
7. **Notify other operators** with the new chainId, the routers added, and the price feed source used.

13.5 Verifying audit log integrity

When to use: routine (e.g., weekly) or after any incident that might have involved the signer. The hash chain (§ 7.3.4) makes this a cheap, mechanical check.

Steps:

1. **Run the verify script** (no signer restart needed, no unlock needed — the script reads the log file directly):

```
sh tsx scripts/verify-signer-audit.ts --log /var/log/signer/signer-audit.jsonl
```
2. **Expected output on intact chain:**

```
Verifying signer-audit.jsonl... Entries scanned: 1247 Chain intact: YES Genesis entry: 2026-07-14T10:23:01.234Z Last entry: 2026-07-21T09:15:42.891Z Exit: 0
```
3. **On broken chain** (exit non-zero, CI-gatable):

```
Verifying signer-audit.jsonl... Entries scanned: 1247 Chain BROKEN at entry index 832: expected prevHash: 9f3a... actual prevHash: c7e1... entry ts: 2026-07-18T14:22:11.045Z
```

```
entry rpcId: abc-90210 Exit: 1
```

A break means either (a) the file was edited in place (selective tampering — the most common attacker move), or (b) the file was truncated and appended to (a partial rewrite), or (c) entries were inserted (a more sophisticated attack). Treat any break as a security incident: assume the signer host is compromised, halt trading, and investigate.

4. **If you ship the log to a remote sink (Phase 2):** also run the verify script against the remote copy. If local says intact but remote says broken (or vice versa), one of them was tampered with. The remote copy is more trustworthy (an attacker with root on the signer host can rewrite the local file but cannot rewrite the remote sink).

13.6 What to do when the signer crashes

Symptoms: the web process's `signerClient` reports `disconnected`, the dashboard shows "Vault locked — signer unreachable", trading halts.

Steps:

1. **Check the crash log:** `ls -lt logs/signer-crash-*.log | head -1` then read the most recent file. The crash handler (§ 4.3) writes a stack trace synchronously before exit.
2. **Check the audit log:** the last entry before the crash should match the operation that triggered it (if any). `tail -5 logs/signer-audit.jsonl`.
3. **If the crash is reproducible from the trace:** file an issue, include the crash log + the audit log tail, and DO NOT restart the signer until the root cause is fixed. The vault is in LOCKED state — no keys are at risk.
4. **If the crash is a one-off (e.g., OOM, transient FS error):** restart the signer (`sudo systemctl restart signer` or equivalent). Re-enter the passphrase on the signer TTY (§ 9.1). Verify health: `scripts/signer-cli.sh health_check`.
5. **If the signer auto-restarted (§ 9.2) but the auto-restart cap was hit:** the web process has marked the signer as permanently down and halted the trading engine. Manual intervention required: investigate the crash logs (you should see 3 crashes in 10 min), fix the root cause, then `sudo systemctl reset-failed signer && sudo systemctl start signer`, re-enter passphrase.
6. **After any signer crash, run § 13.5 (audit log integrity check).** A crash is a moment of elevated risk (the signer process was in an unexpected state); verifying the audit log confirms no entry was lost or forged during the crash.

13.7 Known unresolved case: NAT / Docker userland-proxy collapses `peer.address` (v19.3.2)

Symptom: the per-IP rate limiter (§ 7.3.7) appears to be working (the code is correct, the audit log shows IPs being tracked), but all external connections arrive from the SAME source IP — typically the Docker bridge gateway (`172.17.0.1`), the NAT router's internal interface, or a load-balancer's source-NAT address. The operator and the attacker effectively share one rate-limit bucket, recreating the self-DoS condition the v19.3.1/v19.3.2 hotfix was supposed to fix.

Root cause: when the Next.js process runs behind network-level address translation (NAT) or a Docker userland-proxy (`docker run -p 3000:3000` without `--net-work host`) without an HTTP-level reverse proxy terminating the connection first, the kernel's TCP socket peer address reported by `connection().peer.address` is the GATEWAY IP, not the real client IP. The TCP handshake terminates at the gateway; the gateway then opens a new TCP connection to the app. From the app's perspective, every request comes from the gateway.

This is a deployment-environment limitation, not a code bug. The `peer.address` value is correctly reported — it's just that the "peer" is the gateway, not the client.

Detection: check the audit log for the distribution of `sourceIp` values. If 100% of unlocks come from a single IP (especially a private range like `172.17.0.1`, `192.168.0.1`, or `10.0.0.1`), you are behind NAT/userland-proxy and per-IP isolation is NOT in effect for external clients.

```
sqlite3 db/custom.db "SELECT JSON_EXTRACT(context, '$.sourceIp') AS ip, \
COUNT(*) FROM AppLog WHERE source='vault' GROUP BY ip ORDER BY 2 DESC"
```

Mitigation options (in order of preference):

1. **Put Caddy/nginx in front of the app** and configure the shared-secret trust model from § 7.3.7. The proxy terminates TCP, reads the real client IP from the TCP peer, and forwards it via `X-Forwarded-For` + `X-Internal-Proxy-Secret`. The app then trusts XFF (because the secret matches) and per-IP isolation works for real client IPs. This is the only fully-correct fix.
2. **Use Docker `--network host`** (Linux only). The container shares the host's network namespace, so the kernel sees real client IPs directly. `peer.address` is the real client IP. Caveat: this removes Docker's network isolation — only acceptable if the container is otherwise trusted.

3. Accept the degradation and rely on the global aggregate cap.

The per-IP rate limiter degrades to a shared bucket in this case, but the GLOBAL aggregate cap (50 failures/1h across ALL IPs, a single array NOT subject to per-IP Map eviction) still catches distributed attacks, because it is independent of the per-IP key. The operator will see `vault_rate_limited_global` events when the aggregate threshold is crossed — this is the safety net. Caveat: the per-IP cooldown (5min for one IP) is unavailable, so a single determined attacker can hammer at 50 attempts/hour indefinitely before the global cap re-fires every hour.

What does NOT work:

- **Binding to a different interface:** the issue isn't the bind address, it's the network path. The kernel sees the gateway because the gateway is the TCP peer — changing the bind address doesn't change that.
- **Trusting `x-forwarded-for` without a shared secret:** this recreates the v18 bypass-the-rate-limiter bug (XFF is client-writable, attacker rotates a fake value per request). NEVER do this. The shared secret in § 7.3.7 is non-negotiable.

Action: if you cannot deploy option 1 or 2, document the deployment as "per-IP rate limiter NOT in effect — relying on global aggregate cap only" in the operator runbook for this environment. Increase monitoring on `vault_rate_limited_global` events — they are the only defense layer active in this topology.

13.8 Detecting silent degradation of the peer-address capture (v19.3.2-review, draft-6)

Symptom: the per-IP rate limiter (§ 7.3.7) silently stops capturing real TCP peer addresses. All requests — including direct connections that SHOULD report distinct IPs — collapse to the `"unidentifiable-socket"` bucket. The rate limiter does NOT crash; it keeps running, but the per-IP key for every request is the same string, recreating the v19.3.1 self-DoS shape under a different bucket name.

Root cause: the peer-address capture in `src/lib/request-peer-capture.ts` is implemented as a monkey-patch on `http.Server.prototype.emit('request', ...)`. This coupling is explicit (see the file header comment). The patch silently does nothing if requests stop flowing through that specific emit path. Known cases where this happens:

1. **HTTP/2 server via `http2.createSecureServer()` directly.** HTTP/2 servers emit a `'stream'` event, not `'request'`. The patch listens

- for `'request'` only. No crash, no warning — just no capture.
2. **Custom server adapter (e.g., Fastify mounting Next.js).** If the adapter re-emits requests through a different event or wraps `IncomingMessage` before emitting, the patch may run but `req.socket` may be absent — `peerAddress` falls back to null.
 3. **Serverless platform that synthesizes `req` objects from cloud events (e.g., AWS Lambda adapter for Next.js).** The synthetic req has no real socket — `peerAddress` is null.
 4. **Next.js Edge runtime.** The `instrumentation.ts` guard (`process.env.NEXT_RUNTIME !== 'nodejs'`) skips the patch installation entirely on Edge — correct, because Edge has no `http.Server` to patch, but it means `getRequestPeerAddress()` always returns null in that runtime.
 5. **The try/catch safe-degradation path firing repeatedly.** If a runtime bug causes the peer-address capture to throw on every request (e.g., a corrupted ALS instance, a V8 async-hooks stack overflow), the patch catches the error, falls back to null, and writes a warning to stderr — but the request still completes. The server stays up; per-IP isolation silently degrades.

Detection: query the audit log for entries with `sourceIp = 'unidentifiable-socket'`. In a healthy direct-connection deployment (the project's actual topology since v18), this count SHOULD be zero. A non-zero count means the capture mechanism is NOT working — investigate the runtime / server adapter before treating the rate limiter as functional.

```
sqlite3 db/custom.db "SELECT COUNT(*) FROM AppLog \
WHERE source='vault' AND JSON_EXTRACT(context, '$.sourceIp') = 'unidentifiable-socket'"
```

For continuous monitoring, alert if this counter is non-zero for more than 5 minutes in a deployment that expects direct connections (no proxy configured, no NAT/Docker userland-proxy — see § 13.7 for that case).

Mitigation:

1. **Stay on `http.Server` (the default Next.js Node.js runtime path).** This is the deployment the project has used since v18. The patch works; no action needed.
2. **If migrating to HTTP/2 or a custom server adapter:** re-implement peer-address capture at whatever layer sees the raw TCP socket. For HTTP/2, that's the `'stream'` event handler's `session.socket.remoteAddress`. For a custom server, it's whatever the adapter exposes. The pure decision function

`resolveTrustedClientIp` does NOT change — only the capture mechanism does.

3. **If deploying to serverless / Edge:** accept that per-IP rate limiting is NOT functional in that environment. Rely on the global aggregate cap (50 failures/1h) as the only defense layer. Document this explicitly in the deployment runbook — the operator must know per-IP isolation is off before relying on it.

Action: before treating the rate limiter as functional in any deployment, run the detection query above. If the count is non-zero, the capture mechanism is degraded — investigate before relying on per-IP isolation. This check is part of the operator runbook for a reason: the failure mode is SILENT (no crash, no error log entry beyond the stderr warning), which is exactly the kind of gap this investigation has been trying to close.

14. Summary

This design moves the wallet vault into a dedicated child process that communicates with the Next.js web process via a Unix domain socket using JSON-RPC 2.0 with a strict method allowlist. The key security property is that decrypted private keys never exist in the web process's memory — a compromise of the web process (the most exposed component) cannot directly access keys, and a crash of the web process causes the signer to immediately zero all keys.

v19.3 update (draft-2): the original draft of this design treated the signer as a pure signing oracle — process isolation without a trust boundary. The operator's review identified this as the critical gap: a compromised web process can still send arbitrary `sign` RPCs through the legitimate IPC channel, and the `hotWalletCapUsd` check in `portfolio.ts` is on the wrong side of the trust boundary to stop it. The revised design adds **§ 7.3 Signer-Side Authorization Policy** as a Phase 1 BLOCKING prerequisite: per-transaction value cap, per-time-window rolling cap, destination address allowlist, and an independent signer-written audit log. The signer is now a real trust boundary, not just a process boundary. The § 9 answers are tightened to the operator's stated defaults (passphrase never crosses IPC, fail-closed restart, symlink-swap check on socket parent dir, explicit single-writer, minimal pinned deps).

v19.3-design-draft-3 update (operator's conditional approval round): five sign-off prerequisites incorporated:

1. **§ 9.4 writer lease with TTL** — the writer lock is now a lease that auto-expires (default 30s) if the holder stops renewing (default renew every 10s). On expiry, the signer zeroizes keys and goes LOCKED — same fail-closed consequence § 9.2 already accepts for restart, now explicit as a designed property of the lock, not an accident.
2. **§ 7.3.4 hash-chained audit log moved to Phase 1** — local tamper-evidence is a few lines of code on the write side + a linear scan on the verify side. Remote shipping (syslog/Loki/S3) stays Phase 2. The `scripts/verify-signer-audit.ts` script is CI-gatable.
3. **§ 7.3.7 failed-unlock rate limiter is now per source IP** — was global, which created a self-inflicted DoS (attacker from one IP blocks operator from another). The 150s stress test data confirmed the bug was manifesting: the `attacker` profile's 5 wrong-passphrase failures triggered a 5min cooldown that blocked the `operator` and `lifecycle` profiles for the remaining ~2min of the test. The USD rolling cap (§ 7.3.2) and per-tx cap (§ 7.3.1) remain global — the signer is single-writer, so per-IP scoping would weaken them without adding defense.
4. **§ 7.3.1 price feed source is fixed at boot** — explicit confirmation that the URL/source is bound to env vars at signer boot, NOT a parameter of the `sign` RPC. A `sign` RPC carrying `priceFeedUrl` or `price` is rejected with `-32602 Invalid params`. Same tamper-resistance pattern as the allowlist and the value caps.
5. **§ 13 Operator Runbook** — step-by-step procedures for adding/removing allowlist entries, changing caps, expanding to new chains, verifying audit log integrity, and recovering from signer crashes. Every procedure that touches signer-side config involves restart + re-unlock — that is the designed friction, now documented as procedure rather than implied by design.

The design is deliberately conservative in what it reuses (existing vault class, crash handler) and deliberately aggressive in what it adds (signer-side policy, independent audit log with hash chain, writer lease with TTL, TTY-mode passphrase, per-IP rate limit). The only new surface is the IPC protocol, which is a standard (JSON-RPC 2.0) with a hard-coded method allowlist plus the § 7.3 policy enforced before any sign operation reaches the key. Phase 1 is now a 7-8 day implementation (up from 6-7 in draft-2) — the extra day covers the writer lease + TTL + zeroize-on-expiry, the hash chain + verify script, the per-IP rate limit refactor, and the runbook. Phase 2 hardening (HMAC request signing, signer-side notifier, remote audit log shipping) is deferred.

The crash investigation checks ran in parallel with this design revision:

- **CHECK 1 (clean 150s heterogeneous run, v19.3):** NEGATIVE — 247 cycles, zero organic crashes, zero new crash-.log during the stress loop.
- **CHECK 2 (crash handler armed under load, v19.2):** PASSED — handler fires under concurrent load, produces crash-.log with full stack trace.
- **Port-collision hypothesis (v19.3):** INDETERMINATE — no direct EADDRINUSE evidence, but hypothesis structurally plausible and consistent with "doesn't reproduce in production". New `scripts/check-port-orphan.sh` helper closes the operational hygiene gap.
- **Per-IP rate limit bug (v19.3-design-draft-3):** CONFIRMED — the 150s stress test data shows the `attacker` profile's cooldown blocked the `operator`/`lifecycle`/`interleaver` profiles. The bug was already in our own evidence; we just hadn't framed it that way. Originally scoped as Phase 1; per operator directive, moved to independent hotfix (v19.3.1, then v19.3.2 fix).
- **v19.3.1 sentinel bug (v19.3-design-draft-5):** CONFIRMED — the v19.3.1 hotfix's `"direct-untrusted"` fallback sentinel was a single shared bucket for ALL direct connections. Since this project never runs behind a reverse proxy (bind `127.0.0.1` since v18 is the primary security posture), every real connection fell into the shared sentinel bucket — the hotfix did not fix the bug for the deployment running today. v19.3.2 replaces the sentinel with the actual TCP socket peer address, captured via an `http.Server.prototype.emit` monkey-patch + `AsyncLocalStorage` (Next.js 16 App Router does NOT expose `req.socket.remoteAddress` to route handlers — `connection()` from `next/headers` returns `Promise<void>` in 16; the `peer.address` API from 15.3 was removed). This gives real per-IP isolation for the actual deployment. Regression test scenario 18 proves two distinct direct IPs do NOT share a bucket (via the pure decision function); scenario 19 proves forged XFF without proxy secret is ignored. **However**, draft-5's claim that "v19.3.2 is shipped, Phase 1 ready without reservations" was premature — see the v19.3.2-review item below.
- **v19.3.2 monkey-patch review (v19.3-design-draft-6):** operator review of draft-5 caught three remaining issues in § 7.3.7 that draft-5 had not addressed: (1) the `patchedEmit` function in `request-peer-capture.ts` did NOT wrap the peer-address capture nor the `runWithPeerAddress` call in try/catch — a single exception in either would propagate up through `http.Server.prototype.emit` and crash the server on EVERY request, a strictly worse failure mode than the silent-crash bug being investigated; (2) scenarios 18 and 19 in `test-vault.ts` called the PURE decision function with synthetic peer-address values — they did NOT exercise the actual monkey-patch nor the `AsyncLocalStorage` propagation, leaving the

riskiest component untested (the same pattern that bit this thread three times already); (3) the coupling of the capture mechanism to `http.Server.prototype.emit('request', ...)` was not documented, meaning a future migration to HTTP/2 or a custom server would silently degrade the rate limiter to the shared `"unidentifiable-socket"` bucket. Draft-6 addresses all three: the monkey-patch now has try/catch with documented fallback, the new `scripts/test-request-peer-integration.ts` exercises the REAL monkey-patch + ALS with a live `http.Server` and two distinct loopback source IPs under concurrent requests-in-flight, and the `http.Server` coupling is documented in `request-peer-capture.ts` with detection query in § 13.8. The rate limiter hotfix is now genuinely ready for operator re-review.

The silent-crash investigation remains at **MONITOR**. The crash-logger is armed, the heterogeneous recipe does not reproduce in production build, and the next time any silent crash recurs the crash log will give a real stack trace. **Phase 1 implementation status — split:**

- **APPROVED, ready to begin:** § 7.3.1 (price feed fixed at boot), § 7.3.4 (hash chain + anti-truncation checkpoint), § 9.4 (writer lease with TTL + immediate release on clean disconnect + `writer_lease_released` vs `writer_lease_expired` distinction), § 13 (full operator runbook).

- **NOT YET APPROVED — awaiting operator re-review of draft-6:** § 7.3.7 (rate limiter hotfix v19.3.2). The three issues the operator identified in draft-5 are addressed in draft-6, but the operator has not yet seen the evidence. The try/catch diff, the integration test results, and the `http.Server` coupling documentation are all in place — but per the operator's explicit directive ("Preciso que isso seja corrigido antes de eu considerar o item 3 fechado" and the draft-6 review note that treating an unverified claim as approval is exactly the kind of gap this discipline exists to catch), Phase 1 is NOT yet "ready without reservations". It will be once the operator confirms § 7.3.7 is closed on the draft-6 evidence.